
PILLAR 09

Scale-to-Zero Architecture & Analytics

The stack, the data model, and the true cost of idle.

Respect the Funk · marketing engine research program

Target: *Losing Sleep* — Hallow Youth · 15 July 2026

1,208 lines · 331 source links

Pillar 9 — Scale-to-Zero Architecture, Analytics Ingestion & Cost Attribution

Research date: July 2026 **Scope:** data architecture, ingestion/warehouse layer, cost attribution/FinOps, and the scale-to-zero pattern itself. **Out of scope:** unit pricing of individual compute/AI/storage components (see Pillar 7 — cost model).

Bottom line / recommendation

Recommended stack (Tier 1, "\$0-at-idle scrappy"):

LAYER	PICK	IDLE COST
Compute / app	Cloudflare Workers (Free) or Vercel Hobby	\$0
OLTP database	Cloudflare D1 (Free) or Neon Free	\$0
Cache / queue	Upstash Redis (PAYG) or Cloudflare Queues	\$0
Warehouse	Nothing yet — the OLTP Postgres/D1 is the warehouse	\$0
ELT	<code>cron</code> + a <code>dlt</code> Python script hitting Meta + TikTok APIs	\$0
Orchestration	Vercel Cron / Cloudflare Cron Triggers	\$0
Cost ledger	A <code>cost_event</code> table you write yourself	\$0
ML	None. Use Meta Advantage+ / TikTok Smart+	\$0

Honest idle cost: \$0/mo is achievable, but only on a narrow path. The `$0` claim survives only if you (a) stay on free tiers, (b) never attach a VPC, and (c) accept Vercel Hobby's once-per-day cron limit. Realistically, a production deployment lands at **\$0–\$25/mo**, and the moment you want a team seat on Vercel Pro (\$20/user/mo) or Supabase Pro (\$25/mo), you have a permanent floor that no amount of "serverless" changes.

The three findings that matter most:

- Networking sidecars are what kill the zero, not compute or databases.** AWS Lambda and Cloud Run are genuinely \$0 at idle *in isolation*. Attach them to a VPC-resident database and you're forced onto a NAT Gateway (~\$33/mo, [AWS VPC pricing](#)) or a Serverless VPC Access connector with a 2-instance minimum whose charges, per Google's own docs, "persist regardless of whether your service is actively receiving requests" ([Cloud Run: connecting to a VPC](#)). **Architectural rule: only use databases reachable over public HTTPS** (Neon, D1, Upstash REST, Turso) so the VPC tax never applies.
- "Scale to zero" means three incompatible things and two of them aren't zero.** Neon suspends *compute* to \$0 but bills storage forever. Supabase Free reaches true \$0 by *breaking* — it pauses after 7 days and needs a manual dashboard restore ([Supabase free project pausing](#)) — which disqualifies it for scheduled ad webhooks; the only always-available Supabase option is Pro at a flat \$25/mo that **never** scales down ([Supabase pricing](#)). PlanetScale simply doesn't offer scale-to-zero and has had no free tier since April 2024 ([PlanetScale pricing](#)).

3. **Almost the entire "modern data stack" is unjustified at your scale.** You have ~3 data sources and will produce single-digit GB/year. Skip: Snowplow, Segment, Fivetran, Airbyte, Snowflake, Databricks, ClickHouse Cloud, Iceberg/Delta, Feast, and all custom ML. Each solves a scale problem you do not have, and several impose a monthly floor regardless of usage. **The simplest thing that works is a cron job, a Python script, and one Postgres database.**

Hard blocker to know now: Spotify for Artists has no public API. Stream data is a manual CSV export or a paid third-party aggregator (Chartmetric, ~\$350/user/mo), not an "add a connector" task. This is the single largest constraint on the whole analytics design — see §2D.

Table of contents

1. [Scale-to-zero, honestly assessed](#)
2. [The event/analytics pipeline](#)
3. [The data model](#)
4. [Cost tracking / FinOps](#)
5. [ML layer, scale-to-zero](#)
6. [Multi-tenancy](#)
7. [Reference architectures](#)
8. [Addendum — independent verification pass \(July 2026\)](#) — ⚠ contains **two corrections** to §4.2 and §5.3
9. [Sources](#)

1. Scale-to-zero, honestly assessed

1.1 Compute

COMPONENT	TRULY ZERO-AT-IDLE?	COLD START	REAL COST AT IDLE (MONTHLY)	SOURCE
Vercel Functions / Fluid Compute	It's complicated — Active CPU billing means \$0 between requests, but production usually pulls you onto Pro	Reduced by warm-instance reuse; no official number published	\$0 on Hobby; \$20/user/mo flat on Pro (incl. \$20 usage credit). The floor is the <i>plan</i> , not the compute	Vercel functions pricing , Vercel pricing
Cloudflare Workers	Yes on Free	Sub-ms to low-ms (V8 isolates, not containers); precise 2026 figure UNVERIFIED	\$0 on Free (100K req/day, 10ms CPU/invocation); \$5/mo only if you need Paid (Durable Objects, higher limits)	Workers pricing
AWS Lambda (no VPC)	Yes	100ms–1s+ by runtime	\$0 (1M req/mo always-free, then \$0.20/M + duration)	Lambda pricing

COMPONENT	TRULY ZERO-AT-IDLE?	COLD START	REAL COST AT IDLE (MONTHLY)	SOURCE
AWS Lambda + VPC + NAT Gateway	No — hard floor	VPC penalty is now ~50ms—few hundred ms via Hyperplane ENIs, <i>not</i> the legacy 10–15s	~\$33/mo (\$0.045/hr × 730) plus \$0.045/GB, billed whether or not Lambda ever fires	VPC pricing, Lambda cold starts
Modal	Yes — per-second, "never pay for idle"	Docs cite 2–4s container warm-up	\$0 on Starter (+\$30/mo free credits). Gotchas: <code>min_containers > 0</code> creates a floor; Team plan is \$250/mo base	Modal cold start, Modal pricing
Google Cloud Run (min-instances=0, Direct VPC egress)	Yes	Node/Python ~500ms–2s; Java/.NET 3–10s; GPU 5–20s — all third-party, Google publishes no hard number	\$0. Free tier is region-locked to us-central1/us-east1/us-west1	Cloud Run pricing, min-instances
Cloud Run + legacy Serverless VPC Connector	No — hidden floor	same	Minimum 2 always-on connector instances billed as Compute Engine VMs; charges "persist regardless of whether your service is actively receiving requests." ~\$15–20/mo is a third-party estimate — UNVERIFIED ; official pricing is per-throughput-unit-hour	connecting to a VPC, Serverless VPC Access
Fly.io Machines (auto-stop, no volume/dedicated IP)	Yes — stopped machines bill \$0 compute	No published number; Firecracker restore typically sub-second — UNVERIFIED	\$0 (shared IPv4 + Anycast IPv6 free)	Fly.io pricing
Fly.io + volume or dedicated IP	No — floor	same	Volumes bill \$0.15/GB-mo continuously even when the machine is stopped (10GB = \$1.50/mo); dedicated IPv4 \$2/mo flat	Fly.io pricing, Fly.io cost management

1.2 Databases — where the marketing lies

COMPONENT	TRULY ZERO-AT-IDLE?	COLD START	REAL COST AT IDLE (MONTHLY)	SOURCE
Neon (Free)	Yes , mandatory autosuspend	Docs say "a few hundred ms"; independent benchmarks report median ~1.8s / p95 2.6s	\$0 (100 CU-hrs, 0.5GB storage)	Neon scale-to-zero, Neon latency benchmarks

COMPONENT	TRULY ZERO-AT-IDLE?	COLD START	REAL COST AT IDLE (MONTHLY)	SOURCE
Neon (Launch/Scale)	It's complicated	same	Compute \$0.106/CU-hr (Launch) / \$0.222 (Scale), \$0 when suspended; storage bills continuously at \$0.35/GB-mo. ⚠️ A Neon GitHub discussion documents a <code>check_availability</code> control-plane heartbeat pinning ~0.25 CU on a low-traffic Launch project — reportedly ≈ \$19/mo "minimum effective cost." UNVERIFIED (single source) — test empirically before relying on \$0	Neon pricing, neon discussion #12900
Supabase (Free)	Yes, destructively	N/A — pauses after 7 days inactivity , needs manual dashboard restore (90-day retention)	\$0, but pausing is disqualifying for scheduled ad webhooks	Supabase free project pausing
Supabase (Pro)	No — never pauses	N/A, always-on	Flat \$25/mo floor regardless of traffic — Supabase's own pricing page says pausing: "Never"	Supabase pricing
PlanetScale	No — the feature doesn't exist	N/A	\$5/mo (Postgres PS-5 single node), \$15/mo 3-node HA. Free tier removed April 2024, never returned	PlanetScale pricing
Turso / libSQL	Yes on Free — idle DBs are files in object storage; no process to run	UNVERIFIED; embedded replicas serve reads locally, largely sidestepping the concept	\$0 (5GB, 100 DBs, 500M row reads/10M writes per mo). Developer plan is a flat \$4.99/mo — paid tiers are subscriptions, not metered-to-zero	Turso pricing
Cloudflare D1	Yes — the cleanest story in this audit . Docs explicitly state scale-to-zero with no compute charges at all	N/A (SQLite; no suspend/resume cycle)	\$0 on Workers Free (5M rows read/day, 100K written/day, 5GB). Workers Paid \$5/mo only raises caps — it is not required to use D1	D1 pricing
DuckDB (embedded)	Trivially yes — a library, not a service	N/A	\$0; cost is whatever hosts it	DuckDB docs
MotherDuck	It's complicated	~100ms "Duckling" spin-up — UNVERIFIED (marketing claim, not in pricing docs)	\$0 possible on Free (10 compute-hrs, 10GB storage), but a 1–10 min cooldown after your last query is billable warm time ; storage \$0.04/GB-mo continuous. Next tier reportedly \$250/mo org floor (10x'd from \$25 in the Dec 2025–Feb 2026 repricing — third-party report, verify directly)	MotherDuck billing, MotherDuck pricing
Upstash Redis (PAYG)	Yes for requests; storage is the asterisk	N/A — REST-based, no persistent connection needed	\$0 within free tier (256MB, 500K commands/mo); then \$0.20/100K commands + \$0.25/GB storage that bills with zero traffic if data is persisted	Upstash pricing

COMPONENT	TRULY ZERO-AT-IDLE?	COLD START	REAL COST AT IDLE (MONTHLY)	SOURCE
Upstash Redis (Fixed plans)	No — floor by design	N/A	\$10–\$1500/mo flat. Easy to select by accident instead of PAYG	Upstash pricing

1.3 The gotchas, stated plainly

Gotcha #1 — the VPC tax. This is the "\$60/mo hidden floor" pattern in its most common form. A team picks Lambda ("scales to zero!") and RDS/Cloud SQL, then discovers the NAT Gateway at ~\$33/mo ([AWS VPC pricing](#)) plus an always-on RDS instance, and the "serverless" bill is \$60–100/mo before a single request. Neither the NAT Gateway nor the VPC connector appears on the serverless product's own pricing page. **Mitigation: HTTPS-reachable databases only.**

Gotcha #2 — "zero" is never total cost. Every compute-suspending database still bills storage. Neon's scale-to-zero suspends compute; storage continues at \$0.35/GB-mo ([Neon pricing](#)). This is fine at our scale (a few GB = ~\$1/mo) but it means "\$0 at idle" is marketing shorthand, not an invoice.

Gotcha #3 — the plan floor beats the usage floor. Vercel Pro is \$20/user/mo and Supabase Pro is \$25/mo. Both are *flat* — they do not scale down, ever. For a team of two on Vercel Pro + Supabase Pro, you have a **\$65/mo floor** on an architecture whose compute genuinely costs \$0 at idle. This is exactly the trap the brief asked to catch: the per-seat and per-project plan fees, not the metered compute, are what make a "\$0 stack" cost \$60/mo.

Gotcha #4 — Cloud Run's `min-instances=0` is only free on request-based billing. The scale-to-zero story depends on a billing-mode radio button that has nothing to do with the instance count. Google's own docs are explicit: under request-based billing, "If min instances is set to 0, you are not billed when instances are idle" — but under **instance-based** billing, "If min instances is set to 0, you are **still billed the default rate**" ([Cloud Run: min-instances](#)). Instance-based billing is the mode you select when you need CPU outside of requests (background work, streaming). So the trap is that the exact reason a team switches modes — "our job does work between requests" — is what silently converts a scale-to-zero service into an always-on VM bill, with `min-instances` still reading 0 in the console. **Mitigation: treat the billing mode as a code-review gate, not a deploy-time detail.**

Gotcha #5 — pricing volatility is a real risk for a reusable template. MotherDuck's second tier reportedly went **\$25 → \$250/mo in one cycle** (third-party report — verify directly), and Dagster removed free credits from Solo/Starter in May 2026 ([Dagster pricing updates](#)). For infrastructure meant to be redeployed across many artist/label clients, vendor repricing is itself an architectural risk — prefer components with a credible free tier *and* an open-source escape hatch.

1.4 Verify-before-committing list

1. Neon's `check_availability` heartbeat (~\$19/mo). Single-source and consequential. Deploy a throwaway Launch project, leave it idle a week, read the invoice.
2. MotherDuck's \$250 tier and the reported 10x hike. Third-party report only.
3. The ~\$15–20/mo Cloud Run VPC connector estimate. Official pricing is per-throughput-unit-hour, so actual cost is configuration-dependent.

2. The event/analytics pipeline

Framing: you will produce, generously, low-millions of rows per year. Meta and TikTok report *aggregates* via API — you are not ingesting raw impression firehoses from them; you are pulling daily rollups. The only genuinely event-level, per-user data you own is smart-link clicks. **This is a small-data problem wearing a big-data costume.**

A) Ingestion

OPTION	MODEL	FREE TIER / ENTRY COST	SELF-HOST	VERDICT AT OUR SCALE
Direct-to-warehouse (app writes to Postgres)	none	\$0	N/A	✅ Right-sized
PostHog	usage-based	1M events/mo free, then \$0.00005/event	Yes, MIT (self-host docs)	Fine on free tier if you want session/product analytics; not needed for ad data (PostHog pricing)
Snowplow	BDP Cloud (contract) or Community Edition	BDP entry ~\$1,000/mo+ (quote-based) — UNVERIFIED , third-party	Community Edition free, but real infra cost	❌ Overkill. Built for schema-strict behavioral data at 100M+ events/mo
Jitsu	cloud or self-host, MIT	Cloud free to 200k events/mo, ~\$20–40/mo above	Yes, ~\$5–10/mo self-host (Jitsu pricing)	Cheapest real collector if you eventually want one
RudderStack	usage/seat	Free tier exists; exact current cap conflicting across sources — UNVERIFIED (RudderStack pricing)	Yes, OSS warehouse-native	Middle ground; confirm free tier directly before committing
Segment	MTU-based	Free to 1,000 MTUs / 2 destinations; Team \$120/mo for 10k MTUs (Segment MTU docs)	No	❌ Mismatched. MTU pricing is for user-tracking CDPs, not ad-aggregate ingestion
Vercel Queues	per-operation	\$0.60/1M operations, 4 KiB chunks (Vercel Queues pricing)	No	✅ Fine as a durable buffer if on Vercel
AWS SQS	per-request	1M req/mo free permanently , then \$0.40/M (SQS pricing)	No	✅ Effectively free; boring and reliable
Cloudflare Queues	per-operation	10k ops/day free (Workers Free); Paid includes 1M ops/mo then \$0.40/M (Queues pricing)	No	✅ Near-zero cost
Kafka / Redpanda	self-managed or serverless	Redpanda Serverless has a free tier (Redpanda Serverless)	Yes	❌ Massive overkill. Designed for millions of events/sec

Recommendation — Ingestion: write directly to Postgres/D1, optionally buffered through Cloudflare Queues or SQS for webhook durability/retry. A dedicated collector solves identity resolution, destination fan-out, and schema governance at scale — none of which you have.

Upgrade trigger: multiple internal services emitting events needing a shared schema and multi-destination routing, or event volume into the tens of millions/month.

B) Storage / warehouse

WAREHOUSE	PRICING MODEL	COST AT SMALL SCALE	GENUINELY \$0 AT IDLE?
BigQuery (on-demand)	pay-per-TB-scanned	1 TiB query processing/mo free + 10 GB storage/mo free , then \$6.25/TiB scanned (BigQuery pricing)	✅ Yes — verified. No compute provisioned when idle, no minimum monthly fee. Storage-at-rest is the only floor, and the first 10 GB is free
MotherDuck	compute-hrs + storage	Free: 10GB storage, 10 compute-hrs/mo. Paid tier reportedly \$250/mo (MotherDuck pricing)	⚠️ Free tier ~yes; billable cooldown tail after each query
ClickHouse Cloud	compute-unit-hrs + storage	Basic tier from ~\$66/mo baseline — UNVERIFIED , third-party (ClickHouse pricing analysis)	❌ No perpetual free tier; real floor
Snowflake	credits + storage	Illustrative X-Small warehouse 8hr/day ≈ \$440/mo compute — UNVERIFIED , third-party estimate (Snowflake pricing analysis)	Per-second billing w/ 60s minimum per resume — scales down, but never free
Databricks (Serverless SQL)	DBU-based	~\$0.70/DBU (Databricks SQL pricing)	Scales to zero between queries, but built for Spark-scale problems
Plain Postgres (Neon/Supabase)	see §1.2	\$0–25/mo	✅ Yes on Neon Free

"Just use Postgres until it hurts" — where does it actually break?

There is no single row-count cliff. The useful signals:

- **Size:** Postgres handles analytics into the **hundreds of millions of rows** with tuning. Documented pain begins around **~3 billion rows / ~900 GB** on a single fact table; practitioners reach for columnar engines past roughly **1 TB** ([Tiger Data: the Postgres optimization treadmill](#), [VeloDB: when to scale Postgres analytics](#)).
- **Query pattern (the better signal):** if **EXPLAIN** shows sequential scans averaging **>100,000 rows per scan**, or you need ad-hoc distinct-counts across arbitrary dimensions, row-store starts losing to columnar regardless of absolute size ([Tiger Data](#)).
- **Neon-specific caveat:** Neon's disaggregated compute/storage adds a network hop per page fetch, so it degrades *faster* than vanilla Postgres on large sequential scans and big joins — i.e. exactly the reporting/ETL pattern ([Neon vs Supabase architecture](#)). Keep fact tables narrow and pre-aggregate.

Our numbers: a handful of artists × a few ad accounts × daily aggregate pulls ≈ **low millions of rows/year, likely under 10–50 GB for years**. Postgres will not break. You are roughly **three orders of magnitude** below where a warehouse is justified.

Recommendation — Warehouse: stay on Postgres. When aggregation queries start hurting, move to **BigQuery on-demand** — it's the only warehouse in this audit verified to cost literally \$0 until you run real query volume, and it requires no commitment to test.

Upgrade trigger: dashboard/rollup queries taking multiple seconds, or ad-hoc attribution analysis that keeps demanding new indexes.

C) Lakehouse / table formats (Iceberg / Delta)

Verdict: skip entirely. This is the clearest overkill in the entire report.

Iceberg and Delta Lake exist to let *multiple independent compute engines* (Spark, Trino, Flink, Dremio) share petabyte-scale tables on object storage with schema evolution and time travel ([Dremio: Iceberg vs Delta Lake](#)). Adopting them here means running catalog services, compaction jobs, and metadata management to buy optionality you have no plausible use for. A plain table plus a nightly export to R2/S3 gives you the same durability with none of the tax.

Revisit only if: multiple independent compute engines need to read the *same* raw files without duplicating data. That is not this project's shape and likely never will be.

D) ELT — pulling Meta, TikTok, and Spotify

Connector availability for our three specific sources:

SOURCE	META ADS INSIGHTS	TIKTOK MARKETING API	
Airbyte	✅ Facebook Marketing connector, mature (docs)	✅ TikTok Marketing, Business API v1.3 (docs)	❌ None. The "Spotify Ads" connector is for Ad Studio, not S4A analytics; request is an unresolved open discussion (#36110)
Fivetran	✅ Facebook Ads (docs)	✅ TikTok Ads (connector)	❌ None
dlt	✅ Verified facebook_ads source (docs)	✅ TikTok Business API pipeline (docs)	❌ None for S4A (only public Web API / catalog metadata)
Meltano	✅ tap-facebook (hub)	✅ tap-tiktok (hub)	❌ None

⚠️ The Spotify problem — verified: there is no public API for Spotify for Artists stream data.

- Spotify's Web API covers **catalog metadata** (tracks, albums, artist info) — not S4A dashboard streaming/audience/playlist analytics.
- Spotify for Artists officially supports **manual CSV export only** ([Spotify: Exporting data](#)).
- Spotify reportedly tightened Web API access further in Feb 2026 (Development Mode cut to surface metadata; several endpoints removed; Extended Quota now requires a registered business with 250k+ MAU) — **UNVERIFIED**: the primary community thread 403'd on fetch; corroborated only by third-party commentary ([Spotify's API lock-down](#)).
- Paid aggregators (Chartmetric, Soundcharts) resell deeper access; **Chartmetric's API starts ~\$350/user/mo, rate-limited to 1 req/sec** ([Chartmetric API docs](#); pricing via [Music Analytics Tools](#)) — **UNVERIFIED** whether their access is an official partnership or ToS-gray scraping.

This means: no ELT tool solves Spotify for you. You will write custom code regardless. That single fact collapses the managed-ELT business case.

FACTOR	MANAGED (AIRBYTE/FIVETRAN)	CRON + DLT SCRIPT
Cost at our volume	Airbyte Cloud \$10/mo + \$15/M rows (pricing); Fivetran \$5 base + \$2.50/M MAR (usage-based pricing)	\$0 beyond compute
Sources covered	2 of 3	2 of 3 (dlt verified sources) + Spotify as a custom resource
Ops burden	Low (managed)	dlt handles schema evolution, incremental state, pagination
Solves Spotify?	No	No — but it's the same code either way

Recommendation — ELT: cron + a `dlt` Python script. You need bespoke Spotify code no matter what; paying Fivetran/Airbyte to manage only Meta + TikTok fragments the stack rather than simplifying it, and adds recurring cost for something a free script does.

Upgrade trigger: a 4th+ real API source, non-engineers needing to self-serve connectors via UI, or SLA/RBAC requirements.

E) Orchestration

TOOL	MODEL	COST AT SMALL SCALE	VERDICT
Vercel Cron	scheduled function invocation	Included in plan. Hobby = once-per-day cadence only ; Pro = per-minute. Up to 100 cron jobs/project on all plans (Cron usage & pricing , changelog)	✅ Right-sized if on Vercel
Cloudflare Cron Triggers	scheduled Worker	Included on Workers Free	✅ Right-sized if on Cloudflare
Vercel Workflow	durable steps, usage-based	Usage-based (events, data written/retained); Hobby has monthly allowances (Workflows pricing)	✅ Good step up if pulls need multi-step retry/durability
Prefect Cloud	hosted	Hobby free: 1 seat, 5 workflows, 500 min/mo serverless compute (Prefect pricing)	✅ Best free fallback off-Vercel
Dagster+	seat + credits	Solo \$10/mo + \$0.04/credit. As of May 2026, Solo/Starter include no free credits — billed from zero (Dagster pricing update , pricing)	❌ Overkill + asset/lineage concept overhead
Temporal Cloud	actions + storage	Essentials from \$100/mo (Temporal pricing)	❌ Real floor; built for complex distributed durable execution
Managed Airflow (MWAA)	provisioned environment	From ~\$350/mo (mw1.small) (MWAA pricing)	❌ The worst possible fit. Scheduler + webserver + metastore run 24/7 regardless of workload

Recommendation — Orchestration: Vercel Cron or Cloudflare Cron Triggers. Airflow/Dagster's value — DAG-of-DAG dependency management, asset lineage, backfills across dozens of pipelines — does not exist for 3 data sources. MWAA's ~\$350/mo floor for "run 3 API pulls daily" is pure waste.

⚠️ **Note the Hobby constraint:** Vercel Hobby crons fire **once per day**. If you need hourly pulls, that's either Vercel Pro (\$20/user/mo) or Cloudflare Cron Triggers (free, minute-granularity). This is a concrete case where the \$0 path forces a platform choice.

Upgrade trigger: real DAG complexity, backfill/replay requirements, or SLA-grade alerting.

3. The data model

3.1 Design principles

1. **Store platform-native metrics verbatim; normalize in a view, never on ingest.** The metric definitions are *not* comparable (§3.2). Destroying the raw values on write makes the incomparability invisible and permanent.
2. **Grain: one row per (date × ad × platform × attribution_setting).** Meta and TikTok both report daily aggregates; matching that grain avoids fake precision.

3. **Never sum reach.** TikTok explicitly warns campaign-level reach $\neq$ sum of ad-group reach ([TikTok: About reach](#)). Both platforms' reach is *estimated*.
4. **The stream join is a heuristic, not a foreign key.** Be honest in the schema about this (§3.3).

3.2 Where Meta and TikTok are NOT comparable

Hierarchy (structurally similar, differently named):

LEVEL	META	TIKTOK
L1	Campaign (<code>campaign_id</code>) — objective	Campaign (<code>campaign_id</code>) — <code>objective_type</code>
L2	Ad Set (<code>adset_id</code>) — budget, schedule, targeting, bidding	Ad Group (<code>adgroup_id</code>) — budget, targeting, placements, bidding
L3	Ad (<code>ad_id</code>)	Ad (<code>ad_id</code>)
L4	Ad Creative — separate, immutable object (<code>creative_id</code>)	Creative fields live inside the Ad object; no separate immutable creative object confirmed — UNVERIFIED

Sources: [Meta campaign structure](#), [TikTok Marketing API](#). Meta Insights is queryable at 4 levels ([Ads Insights API](#)); current version is **v25.0** (v23.0 EOL June 9, 2026) ([Graph/Marketing API v25.0](#)).

⚠️ TikTok's business-api.tiktok.com/portal/docs is a JS-rendered SPA that resisted automated extraction. The conceptual facts here are sourced from server-rendered ads.tiktok.com/help pages, but **a full enumerated field list should be pulled from an authenticated developer console session before implementation**.

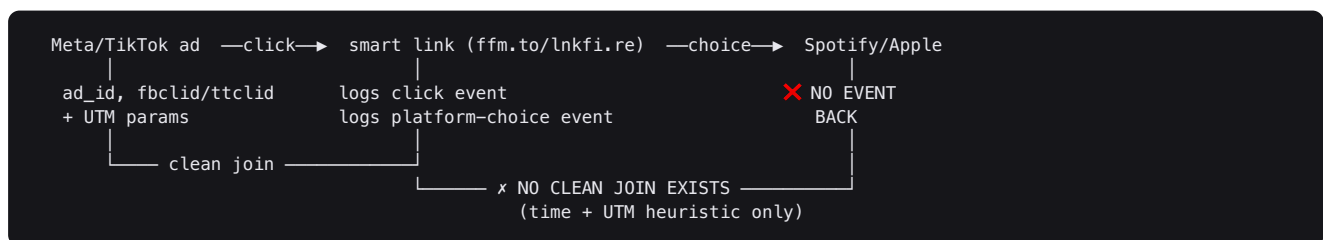
► **Metrics that are NOT directly comparable:**

METRIC	META	TIKTOK	WHY A NAIVE UNION LIES
Video view (mid)	ThruPlay : videos ≤ 15 s counted on completion ($\geq 97\%$); > 15 s counted at 15 continuous seconds (About ThruPlay)	6-second view : ≥ 6 s play, OR played in full if < 6s, OR ≥ 1 engagement within the first 6 seconds (TikTok video play metrics)	Different thresholds (15s vs 6s) and TikTok counts an <i>engagement</i> as equivalent to a view. Meta does not. These are not the same event
Video view (short)	2-second continuous (<code>video_2_sec_watched_actions</code>)	2-second view, replays excluded (source)	Closest pair available, but replay handling differs
View-through attribution	1-day only . 7-day and 28-day view removed Jan 12, 2026 (ppc.land)	1-day or 7-day VTA available (TikTok attribution window)	At platform defaults, TikTok's longer window systematically credits itself more conversions . Must normalize to matching windows before comparing
Engaged view	New 1-day engaged-view window (Mar 2026) — for <i>engagement actions</i> (likes/comments/shares)	EVTA — requires ≥ 6 -second watch (TikTok EVTA)	Similar names, conceptually different triggers
Clicks (all)	<code>clicks</code> — includes Page likes, comments, event responses (Meta: clicks vs link clicks)	<code>clicks (all)</code> — includes social interactions (TikTok basic metrics)	Both are polluted with social clicks. Never use these as "traffic"
Link clicks	<code>inline_link_clicks</code> (fixed 1-day click window internally)	<code>clicks (destination)</code>	✅ This is the comparable pair. Use these

METRIC	META	TIKTOK	WHY A NAIVE UNION LIES
Unique clicks	<code>unique_clicks</code> — counts <i>people</i>	No person-deduplicated click metric found — UNVERIFIED	No like-for-like pair exists. Don't fake one
Reach	estimated; unique-count breakdowns now 13-month retention limit	estimated/sampled; campaign reach $\neq$ sum of ad-group reach (source)	Neither is a census count. Do not sum or compare 1:1
Impressions	total times shown, incl. repeats	"number of times ads were shown"	✅ Most safely unionable — but neither publishes a viewability/render-time spec, so "comparable in concept, not contractually identical"

⚠️ **Retention limits affect backfill design.** As of Jan 12, 2026 Meta limits unique-count/hourly breakdowns to **13 months**, frequency breakdowns to **6 months**, aggregate totals to 37 months ([ppc.land](#)). **You cannot re-pull unique-reach history beyond 13 months.** Ingest early and never delete — your warehouse becomes the only copy.

3.3 The smart-link hop — joining spend to streams



Smart-link tools (Linkfire, Feature.fm, ToneDen) log a **click event** (timestamp, referrer, UTM/click-ID passthrough, device/geo) and a **platform-choice event** naming the chosen DSP ([Linkfire retargeting parameters](#)). Feature.fm exposes an Analytics API with clicks/unique clicks, timeline, geo, and referral breakdowns per SmartLink ([Feature.fm Partners API](#)). ToneDen's FanLinks API supports a `custom target_type` for pixel-tracked redirect with no landing page ([ToneDen FanLinks](#)) — exact click/UTM JSON field names UNVERIFIED.

The honest finding: smart-link vendors do not expose downstream stream events. They get you `ad_click` → `platform_choice`. Actual streaming counts come from Spotify for Artists CSV exports and must be joined **by date + campaign heuristic**, not by an event-level key ([Feature.fm conversion data](#)). This is a genuine data gap that no tool closes. Model it explicitly as a modeled/inferred relationship — never present heuristic attribution as measured fact to a client.

3.4 Proposed schema (PostgreSQL DDL)

```

-- =====
-- CONVENTIONS
-- * All tables carry tenant_id (see §6 — RLS multi-tenancy).
-- * Surrogate keys are internal; platform IDs are natural keys.
-- * Money in NUMERIC(18,6) — never floats.
-- * Platform-native metrics stored verbatim; comparability
--   is a VIEW concern, not an ingest concern.
-- =====

CREATE TYPE channel AS ENUM ('meta', 'tiktok');
CREATE TYPE dsp AS ENUM ('spotify', 'apple_music', 'youtube_music', 'amazon_music', 'other');

-- ----- TENANCY -----
CREATE TABLE tenant (
  tenant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN ('label', 'artist')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ----- DIMENSIONS -----
CREATE TABLE dim_artist (
  artist_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenant,
  name TEXT NOT NULL,
  spotify_artist_uri TEXT, -- from public Web API (catalog metadata only)
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE dim_song (
  song_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenant,
  artist_id UUID NOT NULL REFERENCES dim_artist,
  title TEXT NOT NULL,
  isrc TEXT, -- the only cross-DSP stable identifier worth trusting
  spotify_track_uri TEXT,
  release_date DATE,
  UNIQUE (tenant_id, isrc)
);

CREATE TABLE dim_ad_account (
  ad_account_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenant,
  channel channel NOT NULL,
  platform_account_id TEXT NOT NULL, -- act_id (Meta) / advertiser_id (TikTok)
  currency CHAR(3) NOT NULL,
  timezone TEXT NOT NULL, -- CRITICAL: platforms report in account TZ, not UTC
  UNIQUE (channel, platform_account_id)
);

-- ----- SLOWLY-CHANGING DIMENSIONS (Type 2) -----
-- Campaigns/ad sets are mutable (budgets, names, status change constantly).
-- Type 2 lets "what was the budget on 2026-03-04?" stay answerable.

CREATE TABLE dim_campaign (
  campaign_sk BIGSERIAL PRIMARY KEY, -- surrogate; changes per version
  tenant_id UUID NOT NULL REFERENCES tenant,
  ad_account_id UUID NOT NULL REFERENCES dim_ad_account,
  channel channel NOT NULL,
  platform_campaign_id TEXT NOT NULL, -- natural key, stable across versions
  name TEXT,
  objective_raw TEXT, -- Meta: objective | TikTok: objective_type — NOT normalized
  objective_norm TEXT, -- our best-effort mapping; nullable when unmappable
  status TEXT,
  daily_budget NUMERIC(18,6),
  lifetime_budget NUMERIC(18,6),
  song_id UUID REFERENCES dim_song, -- our own linkage; platforms don't know songs
  valid_from TIMESTAMPTZ NOT NULL,
  valid_to TIMESTAMPTZ, -- NULL = current version
  is_current BOOLEAN NOT NULL DEFAULT true
);
CREATE UNIQUE INDEX ON dim_campaign (channel, platform_campaign_id)
  WHERE is_current;
CREATE INDEX ON dim_campaign (tenant_id, channel, platform_campaign_id, valid_from DESC);

-- Meta "Ad Set" and TikTok "Ad Group" unified into one table.
-- This is a SAFE normalization: the level means the same thing on both
-- platforms (budget + targeting + bidding scope). Metrics are where they diverge.
CREATE TABLE dim_ad_group (
  ad_group_sk BIGSERIAL PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenant,

```

```

channel          channel NOT NULL,
platform_ad_group_id TEXT NOT NULL, -- adset_id (Meta) | adgroup_id (TikTok)
platform_campaign_id TEXT NOT NULL,
name             TEXT,
optimization_goal_raw TEXT,          -- verbatim; deliberately NOT normalized
bid_strategy_raw TEXT,
daily_budget     NUMERIC(18,6),
status           TEXT,
valid_from       TIMESTAMPTZ NOT NULL,
valid_to         TIMESTAMPTZ,
is_current       BOOLEAN NOT NULL DEFAULT true
);
CREATE UNIQUE INDEX ON dim_ad_group (channel, platform_ad_group_id) WHERE is_current;

CREATE TABLE dim_ad (
  ad_sk          BIGSERIAL PRIMARY KEY,
  tenant_id      UUID NOT NULL REFERENCES tenant,
  channel        channel NOT NULL,
  platform_ad_id TEXT NOT NULL,
  platform_ad_group_id TEXT NOT NULL,
  creative_id     UUID,          -- FK added below (dim_creative defined later)
  smart_link_id  UUID,          -- FK added below (dim_smart_link defined later)
  status         TEXT,
  valid_from     TIMESTAMPTZ NOT NULL,
  valid_to       TIMESTAMPTZ,
  is_current     BOOLEAN NOT NULL DEFAULT true
);
CREATE UNIQUE INDEX ON dim_ad (channel, platform_ad_id) WHERE is_current;

-- Creatives are IMMUTABLE on Meta ("you can't change them once created"),
-- so Type 1 is correct here – a "changed" creative is a NEW creative.
-- We keep content_hash to detect the same asset reused across platforms.
CREATE TABLE dim_creative (
  creative_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id      UUID NOT NULL REFERENCES tenant,
  channel        channel NOT NULL,
  platform_creative_id TEXT,      -- NULL on TikTok (no separate creative object)
  media_type     TEXT CHECK (media_type IN ('video','image','carousel','other')),
  duration_sec   NUMERIC(8,2),   -- REQUIRED to interpret ThruPlay (≤15s vs >15s rule)
  content_hash   TEXT,          -- same asset across Meta+TikTok => same hash
  asset_url      TEXT,
  primary_text   TEXT,
  cta_type       TEXT,
  song_id        UUID REFERENCES dim_song,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (channel, platform_creative_id)
);
CREATE INDEX ON dim_creative (tenant_id, content_hash);

CREATE TABLE dim_smart_link (
  smart_link_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id      UUID NOT NULL REFERENCES tenant,
  song_id        UUID NOT NULL REFERENCES dim_song,
  provider       TEXT,          -- linkfire | featurefm | toneden | custom
  url            TEXT NOT NULL,
  utm_campaign   TEXT,          -- our join key back to the ad
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Deferred FKs from dim_ad (declared above, before these tables existed).
ALTER TABLE dim_ad ADD CONSTRAINT dim_ad_creative_fk
  FOREIGN KEY (creative_id) REFERENCES dim_creative;
ALTER TABLE dim_ad ADD CONSTRAINT dim_ad_smart_link_fk
  FOREIGN KEY (smart_link_id) REFERENCES dim_smart_link;

CREATE TABLE dim_date (
  date_key       DATE PRIMARY KEY,
  day_of_week    SMALLINT NOT NULL,
  iso_week       SMALLINT NOT NULL,
  month          SMALLINT NOT NULL,
  year           SMALLINT NOT NULL,
  is_weekend     BOOLEAN NOT NULL
);

-- ----- FACTS -----
-- Grain: one row per (date, ad, channel, attribution_setting).
-- attribution_setting is PART OF THE GRAIN, not an attribute – the same ad-day
-- reports different conversion counts under different windows. Collapsing this
-- is the single most common way marketing warehouses silently lie.
CREATE TABLE fact_ad_performance_daily (
  tenant_id      UUID NOT NULL REFERENCES tenant,
  date_key       DATE NOT NULL REFERENCES dim_date,

```

```

channel          channel NOT NULL,
platform_ad_id   TEXT NOT NULL,
platform_ad_group_id TEXT NOT NULL,
platform_campaign_id TEXT NOT NULL,
ad_account_id    UUID NOT NULL REFERENCES dim_ad_account,
attribution_setting TEXT NOT NULL, -- '7d_click_1d_view' etc. VERBATIM from platform

-- --- SAFE TO COMPARE ACROSS CHANNELS ---
impressions      BIGINT NOT NULL DEFAULT 0,
spend            NUMERIC(18,6) NOT NULL DEFAULT 0,
spend_currency   CHAR(3) NOT NULL,
link_clicks      BIGINT NOT NULL DEFAULT 0, -- Meta inline_link_clicks | TikTok clicks(destination)

-- --- NOT SAFE TO COMPARE: platform-native, stored verbatim ---
clicks_all       BIGINT, -- polluted with social interactions on BOTH platforms
unique_clicks    BIGINT, -- Meta only; TikTok has no equivalent (NULL there)
reach_estimated  BIGINT, -- ESTIMATED on both. NEVER SUM. NEVER COMPARE 1:1.
video_2s_views   BIGINT, -- Meta 2s continuous | TikTok 2s (replays excluded)
meta_thruplays   BIGINT, -- Meta ONLY: ≤15s=completion, >15s=15 continuous sec
tiktok_6s_views  BIGINT, -- TikTok ONLY: 6s OR full-if-<6s OR engagement-in-first-6s
tiktok_15s_focused_views BIGINT, -- TikTok ONLY: opt-in Focused View product
video_p25_views  BIGINT,
video_p50_views  BIGINT,
video_p75_views  BIGINT,
video_p100_views BIGINT,

conversions      BIGINT, -- meaning depends on optimization_goal_raw AND attribution_setting
ingested_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
PRIMARY KEY (tenant_id, date_key, channel, platform_ad_id, attribution_setting)
);
CREATE INDEX ON fact_ad_performance_daily (tenant_id, date_key DESC);
CREATE INDEX ON fact_ad_performance_daily (tenant_id, platform_campaign_id, date_key DESC);

-- Event-level: the ONLY genuinely per-user data we own.
CREATE TABLE fact_smart_link_click (
  click_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id       UUID NOT NULL REFERENCES tenant,
  smart_link_id   UUID NOT NULL REFERENCES dim_smart_link,
  occurred_at     TIMESTAMPTZ NOT NULL,
  channel         channel, -- inferred from click-id presence; NULL if organic
  platform_click_id TEXT, -- fbclid | ttclid
  platform_ad_id  TEXT, -- from UTM passthrough; NULL if not propagated
  utm_source      TEXT,
  utm_campaign    TEXT,
  utm_content     TEXT,
  chosen_dsp      dsp, -- NULL until/unless the platform-choice event fires
  chosen_at       TIMESTAMPTZ,
  country         TEXT,
  user_agent_hash TEXT, -- hashed: don't warehouse raw PII
  ingested_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON fact_smart_link_click (tenant_id, occurred_at DESC);
CREATE INDEX ON fact_smart_link_click (tenant_id, platform_ad_id, occurred_at DESC);

-- Streams: DSP-reported, daily, NO per-user key, NO ad linkage.
-- This table CANNOT be joined to ads on a real key. That is a fact of the
-- ecosystem, not a modeling failure. See fact_attribution_modeled below.
CREATE TABLE fact_stream_daily (
  tenant_id       UUID NOT NULL REFERENCES tenant,
  date_key        DATE NOT NULL REFERENCES dim_date,
  song_id         UUID NOT NULL REFERENCES dim_song,
  dsp            dsp NOT NULL,
  streams         BIGINT NOT NULL DEFAULT 0,
  listeners       BIGINT,
  saves          BIGINT,
  -- NOT NULL with a sentinel: Postgres PKs can't contain expressions, and NULL
  -- country would silently break uniqueness. '--' means "not broken out by country".
  country         TEXT NOT NULL DEFAULT '--',
  source         TEXT NOT NULL, -- 's4a_csv_export' | 'chartmetric_api' | ...
  source_ingested_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, date_key, song_id, dsp, country)
);

-- ⚠️ MODELED, NOT MEASURED. Isolated in its own table so nobody mistakes
-- a heuristic for a measurement. Every row records HOW it was inferred.
CREATE TABLE fact_attribution_modeled (
  tenant_id       UUID NOT NULL REFERENCES tenant,
  date_key        DATE NOT NULL REFERENCES dim_date,
  song_id         UUID NOT NULL REFERENCES dim_song,
  channel         channel NOT NULL,
  platform_campaign_id TEXT NOT NULL,
  attributed_streams NUMERIC(18,4), -- fractional: it's an estimate

```

```

model_name    TEXT NOT NULL,      -- 'click_share_v1' | 'lift_test_v1' | ...
model_version TEXT NOT NULL,
confidence    TEXT CHECK (confidence IN ('low','medium','high')),
computed_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
PRIMARY KEY (tenant_id, date_key, song_id, channel, platform_campaign_id, model_name, model_version)
);

-- ----- COST LEDGER (see §4) -----
CREATE TABLE cost_event (
  cost_event_id BIGSERIAL PRIMARY KEY,
  tenant_id     UUID REFERENCES tenant,      -- NULL = shared/unattributed (be honest!)
  occurred_at   TIMESTAMPTZ NOT NULL,
  category      TEXT NOT NULL,              -- 'llm' | 'compute' | 'storage' | 'ad_spend' | 'saas'
  vendor        TEXT NOT NULL,              -- 'anthropic' | 'vercel' | 'neon' | 'meta' | ...
  sku           TEXT,                       -- model id / function name / plan line
  quantity      NUMERIC(18,6),
  unit          TEXT,                       -- 'tokens' | 'gb_sec' | 'requests' | 'usd'
  cost_usd      NUMERIC(18,6) NOT NULL,
  attribution_method TEXT NOT NULL
  CHECK (attribution_method IN ('direct','proportional','flat_split','unattributed')),
  source        TEXT NOT NULL,              -- 'ai_gateway' | 'vercel_api' | 'manual' | ...
  external_id   TEXT,                       -- vendor request/invoice id for reconciliation
  metadata      JSONB,
  ingested_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON cost_event (tenant_id, occurred_at DESC);
CREATE INDEX ON cost_event (category, occurred_at DESC);
CREATE INDEX ON cost_event (occurred_at DESC) WHERE tenant_id IS NULL; -- the shared bucket

```

The channel-agnostic view — normalizing only what is safe:

```

-- Exposes ONLY cross-channel-safe metrics. Everything definitionally
-- incomparable is deliberately excluded. If a consumer wants ThruPlays,
-- they query the fact table and confront the caveat.
CREATE VIEW v_channel_agnostic_performance AS
SELECT
  f.tenant_id,
  f.date_key,
  f.channel,
  c.song_id,
  f.platform_campaign_id,
  f.attribution_setting,      -- carried through: never silently mixed
  SUM(f.impressions) AS impressions,      -- ✅ comparable
  SUM(f.spend) AS spend,                -- ✅ comparable (same currency required)
  SUM(f.link_clicks) AS link_clicks,      -- ✅ comparable pair
  SUM(f.video_2s_views) AS video_2s_views, -- ⚠️ near-comparable (replay handling differs)
  -- Deliberately absent: reach (estimated, non-additive),
  -- clicks_all (social pollution), thruplays/6s (different definitions),
  -- unique_clicks (no TikTok equivalent).
  CASE WHEN SUM(f.impressions) > 0
    THEN SUM(f.link_clicks)::NUMERIC / SUM(f.impressions) END AS ctr_link,
  CASE WHEN SUM(f.link_clicks) > 0
    THEN SUM(f.spend) / SUM(f.link_clicks) END AS cost_per_link_click
FROM fact_ad_performance_daily f
JOIN dim_campaign c
  ON c.channel = f.channel
 AND c.platform_campaign_id = f.platform_campaign_id
 AND c.is_current
GROUP BY 1,2,3,4,5,6;

```

► The double-count trap — verified empirically, not theorized.

The whole DDL above was executed against PostgreSQL 15.12 and seeded with one ad-day reported under two attribution settings (exactly what Meta returns when you query multiple windows). A naive aggregation that ignores `attribution_setting` does this:

```

-- WRONG: SUM(spend) GROUP BY channel
channel | spend_wrong
-----+-----
meta    | 100.000000
tiktok  | 40.000000

-- RIGHT: pin one attribution_setting
channel | spend_right
-----+-----
meta    | 50.000000  ← actual spend
tiktok  | 40.000000

```

Meta's spend is reported as \$100 when \$50 was actually spent — a 2× overstatement that would halve every ROAS/CPA figure derived from it. The same ad-day legitimately appears once per attribution window; summing across windows double-counts real dollars. Any consumer of `v_channel_agnostic_performance` **must** filter to a single `attribution_setting` (or group by it). This is the most dangerous failure mode in the entire schema, it is silent, and it produces plausible-looking numbers — which is why `attribution_setting` is in the primary key rather than an optional attribute.

Verification status: the DDL, the view, and the RLS policy in §6.2 all execute cleanly on PostgreSQL 15.12, and RLS isolation was confirmed (a non-superuser with a foreign `app.tenant_id` sees 0 rows; the owning tenant sees all 3). The schema is *syntactically* proven; it has **not** been validated against live Meta/TikTok API payloads — do that before relying on the field mappings.

Why this design refuses to normalize certain things: the temptation is a single `video_views` column fed by both platforms. That column would be a lie — TikTok's 6-second view counts an engagement-in-first-6-seconds as a view, Meta's ThruPlay does not, and the thresholds differ (6s vs 15s). A client comparing "TikTok video views vs Meta video views" from such a column would draw a false conclusion. **Storing both verbatim and refusing to merge them is the correct engineering choice**, even though it makes the dashboard harder to build.

3.5 SCD implementation note

Type 2 on campaigns/ad groups, Type 1 on creatives (Meta creatives are immutable by definition — [Meta campaign structure](#)). Daily ELT diffs the current row against the API response and, on change to any tracked column, closes the old version (`valid_to = now()`, `is_current = false`) and inserts a new one. At our volume this is a few hundred rows/day — trivially cheap, and it's what makes "did the CPA change because the creative changed or because the budget changed?" answerable at all.

4. Cost tracking / FinOps

The framing that matters: "the bill" splits into two very different problems. **Pulling our own spend** is largely solved by vendor APIs. **Attributing it per-tenant** is only solved for the costs you tag *at call time* — and it is genuinely unsolvable for some cost categories. An honest per-tenant bill must show its work: which dollars are measured, which are allocated by a proxy, and which are simply unattributable overhead.

4.1 Pulling our own spend

VERCEL

Spend Management ([docs](#), last updated 2026-06-26) does three things at a set spend amount: notify, fire a webhook, and **pause the production deployment of all projects**. The critical details:

- ⚠️ **"Setting a spend amount does not automatically stop usage."** Pausing is opt-in — you must explicitly enable it.
- ⚠️ **It is a soft cap, not a hard cap.** Vercel checks usage "every few minutes," so "notifications, webhooks, and project pausing can trigger several minutes after you cross your spend amount." Their own docs say to "consider setting your spend amount below the absolute maximum you are willing to spend."
- ⚠️ **Scope is narrower than it sounds.** The amount covers *metered resources beyond your plan allocation only* — it **excludes seats, Marketplace integrations, and add-ons**. Exactly the fixed costs that form your real floor (\$1.3) are outside spend management's reach.
- Pausing is **all-or-nothing across the team** — you cannot pause one tenant's project. Visitors get a `503 DEPLOYMENT_PAUSED`.

- **Unpausing is manual**, per project, via dashboard or the [unpause REST endpoint](#). Raising the limit does *not* auto-resume.
- Requires **Owner or Billing role on a Pro team** — not available on Hobby.
- Webhook payload gives `budgetAmount`, `currentSpend`, `teamId`, `thresholdPercent`, fired at 50/75/100%, plus an `endOfBillingCycle` event useful for auto-resuming paused projects.

Verdict: Vercel gives you a real (if delayed and coarse) kill switch at the *team* level. It is a blast-radius limiter, not a per-tenant budget.

AWS COST EXPLORER

- **The API costs money: \$0.01 per request** on your primary billing view; custom billing views cost **\$0.01 per source** (a 5-source view = \$0.05/request) ([AWS Cost Explorer pricing](#)). At \$0.01/call, a naive per-tenant-per-day polling loop across 100 tenants costs ~\$30/mo just to *ask about* the bill — budget the metering.
- **Cost allocation tags** are the attribution mechanism: tag resources with `tenant_id`, then filter/group in Cost Explorer ([Using the Cost Explorer API](#), [ListCostAllocationTags](#)).
- **⚠️ Two operational traps:** a tag key **must be activated** before it's usable in Cost Explorer, and **it can take up to 24 hours** for a tag to appear in billing data. Tags are also **not retroactive** — untagged spend before activation is unattributable forever.

Verdict: irrelevant to the recommended stack (we're on Cloudflare/Vercel). Relevant only if you add AWS resources — in which case, activate the `tenant_id` tag key **before** creating any resource.

CLOUDFLARE

- The [GraphQL Analytics API](#) exposes usage across products; D1 specifically exposes **rows read, rows written, and total storage** across all databases, retained/queryable for **31 days** ([D1 metrics and analytics](#)). Every D1 query returns a `meta` object with `rows_read` and `rows_written` for that query ([D1 billing](#)) — which is the single most useful primitive in this whole section (see §4.3).
- **► Cloudflare's own explicit caveat:** the GraphQL datasets "**should not be used as a measure for usage that Cloudflare uses for billing purposes**" — billable traffic excludes things like DDoS traffic, while GraphQL measures all consumption ([GraphQL Analytics API](#)). **So GraphQL is for attribution ratios, not for reproducing the invoice.** Reconcile against the actual invoice; use GraphQL to split it.
- **⚠️ 31-day retention** means your cost ledger must snapshot this data or lose it.

4.2 LLM cost tracking — comparison

*On "hard cap" in this table: no LLM gateway offers a mathematically exact cutoff. Each checks spend before a request, so the request that crosses the line completes and spend lands slightly over.  below means **enforcing** — subsequent requests are rejected, categorically unlike an alert — with overshoot bounded to roughly one in-flight request. Vercel documents this explicitly ([API Key Budgets](#)); the same race is inherent to LiteLLM's Redis-counter check.*

TOOL	PRICING	PER-TENANT TAGGING	SELF-HOST		BEST FIT
Vercel AI Gateway (Custom Reporting)	Gateway usage + reporting: \$0.075/1,000 tag/user-ID writes and \$5/1,000 queries to the reporting endpoint (custom reporting)	✅ Excellent. <code>user</code> (≤256 chars) + up to 10 tags (1–64 chars each) per request, via <code>providerOptions.gateway</code> or the <code>ai-reporting-user</code> / <code>ai-reporting-tags</code> headers. Query with <code>group_by=user tag model provider credential_type api_key_name</code> , filter by <code>user_id</code> / <code>tags</code> / <code>tags_match</code>	❌	✅ YES — corrected 2026-07. Each API key carries a <code>max</code> spend limit; once hit, the Gateway rejects further requests on that key until the budget resets (daily/weekly/monthly/none) or is raised. Set via dashboard or <code>vercel ai-gateway api-keys create</code> (Budgets for API keys)	✅ Our pick if on Vercel. Returns <code>total_cost</code> , <code>market_cost</code> , <code>input/output/cached/reasoning_tokens</code> , <code>request_count</code> per group — per-tenant LLM cost, essentially for free. Issue one key per tenant and you get attribution <i>and</i> a hard cap from one component
LiteLLM	Open source (proxy); free to self-host	✅ Virtual keys per tenant; spend tracked per key/user/team	✅ Yes	✅ YES. <code>max_budget</code> + <code>budget_duration</code> per virtual key; the proxy rejects requests that would exceed budget , reading spend from a cross-pod Redis counter. Multiple independent budget windows (e.g. \$10/day and \$100/mo) (budgets & rate limits, virtual keys)	⚠️ Secondary — no longer our default. Now that AI Gateway enforces per-key budgets, LiteLLM's main draw is multi-provider routing/fallback. 🛑 Security: PyPI 1.82.7/1.82.8 were backdoored in March 2026 (cloud-credential/SSH/K8s-secret stealer; separate pre-auth SQLi CVE) — pin ≥1.83.0 and audit before giving it credential custody (LiteLLM security update, Trend Micro). See Addendum
Langfuse	Hobby free (50k units/mo, 30-day retention, 2 users); Core \$29/mo (100k units, overage \$8/100k); Pro \$199/mo ; Enterprise \$2,499/mo (Langfuse pricing) — figures via third-party teardown, UNVERIFIED against the live page	✅ Tags/metadata on traces	✅ Yes — MIT. Core features moved to MIT June 2025; self-host free without limits (self-host pricing). Self-host cost driver is ClickHouse (\$200–800/mo) — UNVERIFIED , third-party estimate	❌ Observability, not enforcement	Tracing/debugging depth. ⚠️ ClickHouse acquired Langfuse Jan 2026 — license unchanged so far, but note the ownership change for a reusable template (\$1.3 gotcha #4)

TOOL	PRICING	PER-TENANT TAGGING	SELF-HOST		BEST FIT
Helicone	Free 10k requests/mo (7-day retention); Pro \$79/mo ; Team \$799/mo ; Enterprise custom (Helicone pricing) — via third-party summary, UNVERIFIED	✅ Custom Properties — arbitrary key-value tags (customer ID, feature, environment) become filterable cost dimensions (cost tracking cookbook)	✅ Open source (GitHub)	❌ Tags/filters costs; does not enforce caps	Good tagging model; one-line integration. Redundant if already on AI Gateway
OpenMeter	Usage-based; free to start. Deliberately not revenue-percentage priced (OpenMeter pricing)	✅ Purpose-built: meters arbitrary usage events by subject	✅ Yes — Apache 2.0 ; Go + Kafka + ClickHouse (GitHub)	✅ Supports enforcing limits/entitlements (metering→billing engine, not LLM-specific)	The right tool if you bill clients on usage . Overkill purely for internal cost visibility — it's a billing engine, and it drags Kafka + ClickHouse into a stack whose whole thesis is \$0 at idle

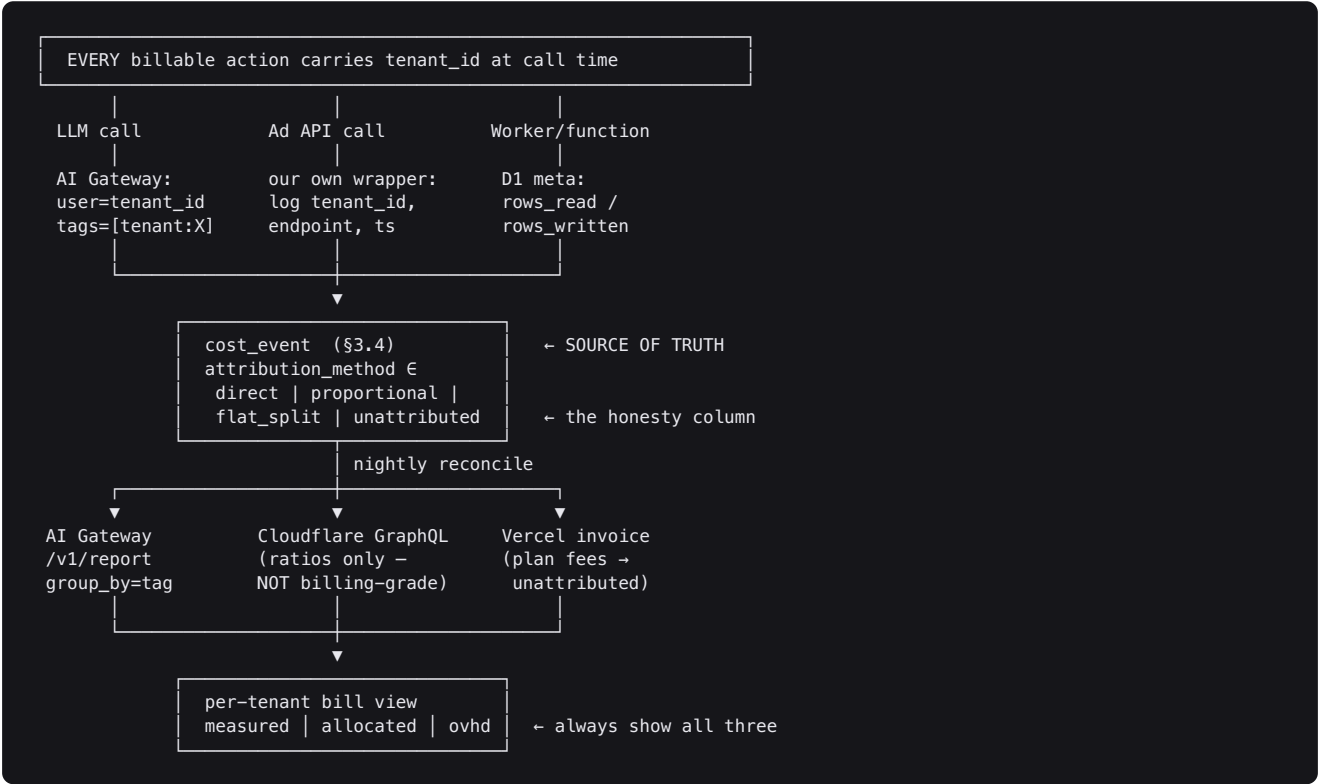
Recommendation (updated 2026-07): Vercel AI Gateway alone — issue one API key per tenant. That single component now gives you *both* attribution (`group_by=tag / user` , essentially free at our volume — 0.075¢ per 1,000 tag writes) *and* a hard cap (per-key `max` spend limit that rejects requests once hit). The earlier "add LiteLLM for enforcement" advice is **superseded**: AI Gateway's budgets close that gap, so a second proxy hop, a Redis dependency, and a fresh credential-custody surface buy nothing. Reach for LiteLLM only for multi-provider routing/fallback AI Gateway doesn't cover — and pin $\geq 1.83.0$ if you do.

Skip Langfuse/Helicone/OpenMeter unless you need deep tracing or usage-based *client* billing. Note the market context: of these four third-party tools, **Langfuse was acquired by ClickHouse (Jan 2026)**, **Helicone is in maintenance mode**, **OpenMeter went to Kong**, and **LiteLLM had a supply-chain compromise** — four-for-four on ownership or security churn in ~18 months. For a template meant to be redeployed across many clients, the native path means fewer moving parts and no third-party credential custody.

⚠️ **Two AI Gateway caveats:** Custom Reporting is **in beta**, **scoped to the entire account** (no per-project key scoping yet), and the reporting endpoint is **Pro/Enterprise only** — **Hobby and Pro-trial cannot use it**. That's a real conflict with the Tier-1 \$0 stack: **per-tenant LLM attribution effectively requires Vercel Pro**. At Tier 1, tag calls anyway and write your own `cost_event` rows from `getGenerationInfo()` / the `generationId` returned on each response.

4.3 Per-tenant attribution architecture

The core discipline: tag at call time, into your own ledger. Vendor APIs are for reconciliation; your `cost_event` table (§3.4) is the source of truth, because only you know what a `tenant_id` is.



The three-bucket rule — every dollar lands in exactly one:

BUCKET	ATTRIBUTION_METHOD	EXAMPLES	HOW
Measured (direct)	direct	LLM calls, ad spend, per-query D1 rows	Tagged at call time. Trustworthy — bill this confidently
Allocated (proxy)	proportional / flat_split	Shared Postgres compute, Workers invocations	Split by a usage proxy. An estimate — label it as one
Overhead	unattributed	Vercel/Supabase seats, plan fees, domains, your time	tenant_id IS NULL . Do not fake a split

The `cost_event` schema (§3.4) enforces this: `attribution_method` is a `CHECK`-constrained column and `tenant_id` is **nullable on purpose**, with a partial index on the shared bucket. **If you cannot honestly attribute a dollar, the schema's answer is `NULL` — not a fabricated allocation.**

Choosing an allocation proxy (best → worst):

1. **Direct metering** — D1's per-query `rows_read` / `rows_written` ([D1 billing](#)) is genuinely per-tenant if you record it per query. This is the best primitive Cloudflare gives you.
2. **Request/invoke count per tenant** — cheap, and a decent proxy for Workers/function cost.
3. **Row count per tenant** — a fine proxy for *storage*, a bad one for compute.
4. **Flat split (÷ N tenants)** — defensible only for genuinely uniform costs. Note it corrodes: at 1 big + 9 tiny tenants, flat split is actively misleading.

What is genuinely hard — stated plainly:

- **Batch LLM calls spanning tenants.** One inference scoring creatives for 5 tenants has *no* honest per-tenant cost — token cost isn't linear in tenant count, shared prompt prefixes benefit from caching (AI Gateway reports `cached_input_tokens` separately, which is exactly the cost you *can't* cleanly divide), and marginal cost per added tenant is lower than average. **Mitigation: don't batch across tenants** while the ledger matters. Per-tenant calls

cost slightly more in tokens and buy you honest attribution. If you must batch, split by token share and mark **proportional** — never **direct**.

- **Shared database compute.** Neon bills CU-hours for a shared instance. A tenant running one expensive analytical query can dominate CU-hours while contributing few rows. Row counts *understate* it, query counts *ignore* cost variance. There is no good proxy short of per-query timing. This is the strongest cost-side argument for branch-per-tenant (§6): it converts an allocation problem into a measurement problem.
- **Plan fees and seats.** Vercel Pro seats, Supabase Pro — these are **the floor from §1.3 and they attribute to nobody**. They're also explicitly outside Vercel Spend Management's scope. Leave them **unattributed**; a tenant-count division is a pricing decision, not a measurement.
- **Platform engineering time.** The largest real cost, and completely outside every API here. Don't pretend otherwise.

4.4 Alerts vs hard caps — what's actually enforceable

MECHANISM	ENFORCEABLE?	LATENCY	GRANULARITY
LiteLLM max_budget per virtual key	✅ Enforcing — proxy rejects further requests (same one-request overshoot as any pre-request check)	Real-time (Redis counter)	Per tenant. ⚠️ Pin ≥1.83.0 — see supply-chain note in §4.2 (docs)
Meta campaign spend_cap	✅ True hard cap — "a hard cap on how much a given campaign spends, regardless of the budget within the adsets under the campaign" (Meta for Business)	Platform-enforced	Per campaign
Meta account spending limit	✅ Hard cap, but cumulative lifetime, not recurring — must be manually reset when reached (Meta: about spending limits)	Platform-enforced	Per ad account
Cloudflare AI Gateway spend limits	✅ Enforcing — requests are blocked by default once the limit is hit. Attribute by tenant via cf-aig-metadata custom attributes (up to 5 fields)	Real-time	Per tenant via metadata; identity-driven per-user/team budgets in closed beta . Only relevant if already on Cloudflare — don't add a second gateway for this (CF blog , Unified Billing)
Vercel AI Gateway API key budgets	✅ Enforcing, with a bounded overshoot. Gateway "checks the budget before each request and rejects further requests once the limit is exceeded." But Vercel also states: "A budget is a soft cap, not a hard limit. The check runs at the start of each request, so the request that crosses the limit still completes." ⇒ overshoot is capped at one in-flight request , not unbounded. ⚠️ New keys are unenforced for 1–2 min after creation; budget <i>changes</i> take tens of seconds to ~5 min	Real-time (pre-request check)	Per key (⇒ per tenant if keyed per tenant) (docs , changelog)
Vercel Spend Management + pause	⚠️ Soft — checks "every few minutes"; usage accrues past the limit	Minutes	Team-wide only — can't pause one tenant (docs)
Vercel spend webhook (50/75/100%)	❌ Alert only — but you can <i>build</i> enforcement on it	Minutes	Team
Cloudflare GraphQL polling	❌ Monitoring only, and explicitly not billing-grade	Poll interval	Per product

MECHANISM	ENFORCEABLE?	LATENCY	GRANULARITY
AWS Budgets / Cost Explorer	❌ Alerting; tags take up to 24h to appear in billing data	Hours–24h	Per tag

The load-bearing insight: the only per-tenant spend enforcement available is at the two edges — the ad platforms' own `spend_cap` (a true hard cap, platform-enforced) and the LLM gateways (Vercel AI Gateway per-key budgets, Cloudflare AI Gateway spend limits, LiteLLM `max_budget` — all enforcing, all with a bounded sub-request overshoot rather than a mathematically exact cutoff). That's convenient, because **ad spend is ~99% of a music-marketing tenant's cost** — and it's the one dollar you *can* cap precisely, per campaign, enforced by Meta itself. Infrastructure spend is the small, hard-to-cap remainder — for it, there is **no per-tenant enforcement primitive at all**, only team-wide blunt instruments and after-the-fact allocation.

Practical posture:

- Per-tenant ad budget:** Meta `spend_cap` per campaign, set at campaign creation. Real cap, real granularity, real money.
- Per-tenant LLM budget:** issue **one AI Gateway API key per tenant** with a budget and refresh period — this gives attribution *and* enforcement from a component already in the stack, no second proxy hop. Reach for LiteLLM only if you need multi-provider routing AI Gateway doesn't cover, and pin $\geq 1.83.0$ if you do (see Addendum: supply-chain compromise).
- Blast radius:** Vercel Spend Management with pausing enabled, set **below** your true pain threshold to absorb the multi-minute lag. Wire the `endOfBillingCycle` webhook to auto-resume paused projects.
- Reconcile nightly:** the ledger against AI Gateway `/v1/report`, Cloudflare GraphQL (ratios only), and the actual invoice. **Snapshot Cloudflare's data — it's gone after 31 days.**

What to build first: the `cost_event` table and the tagging discipline. Both are ~free, and **retroactive attribution is impossible** — untagged spend is unattributable forever (AWS makes this explicit; it's true everywhere). The cost of tagging from day one is a few lines of code; the cost of starting six months late is six months of blind spend you can never allocate to a client.

5. ML layer, scale-to-zero

5.1 Serverless GPU platforms

PLATFORM	COLD START	PRICING MODEL	SCALE-TO-ZERO NOTES	BEST FIT
Modal	~2–4s typical (tens of sec for 70B+ LLMs loading weights)	Per-second: H100 \$0.001097/sec (~\$3.95/hr), A100-80GB \$0.000694/sec, T4 \$0.000164/sec	"You never pay for idle resources — just actual compute time"	Python-native serverless; best DX for a small team (Modal pricing , cold start docs)
Replicate	~5s for public/official models; 60s+ for custom private deployments	A100-80GB \$0.0014/sec (~\$5.04/hr). Public/official models bill only active processing — idle/setup free ; private deployments bill setup+idle+active	Best economics <i>only</i> if using their catalog	Calling existing open models without hosting weights (Replicate pricing)

PLATFORM	COLD START	PRICING MODEL	SCALE-TO-ZERO NOTES	BEST FIT
RunPod (Serverless)	Sub-200ms "FlashBoot" claimed for cached workers; real-world custom containers 20–60s — UNVERIFIED	Per-second, rounded to nearest second; cheapest raw compute of the group. Default 5s idle timeout is billed	True scale-to-zero, configurable idle timeout	Cost-sensitive spiky batch work (RunPod serverless pricing)
Baseten	30–90s for large containers after scale-down; warm pool cuts to 3–8s at ~2x compute cost — UNVERIFIED (third-party)	Per-minute: T4 \$0.01052/min, A100-80GB \$0.06667/min, H100 \$0.10833/min; idle replicas free	Explicit warm-pool tradeoff knob	Production inference with latency SLAs (Baseten pricing)
Beam	Sub-second to 2–3s; cold start / spin-up is not billed	Per-second: H100 \$1.74/hr, A100-80GB \$1.30/hr, RTX 4090 from \$0.69/hr; free tier includes 10 GPU-hrs	Best pure scale-to-zero economics on paper; newer/smaller platform	Early experimentation (Beam pricing)

Observation: scale-to-zero is table stakes for serverless GPU in 2026 — all five do it. The differentiator is cold-start latency vs cost, not capability. For burst batch scoring (nightly creative scoring, not user-facing latency), **Modal** or **RunPod** are the defensible picks.

5.2 Feature stores and experiment tracking

Feast: active, maintained; supports Redis/DynamoDB/Postgres online + BigQuery/Redshift/Snowflake offline ([Feast docs](#), [GitHub](#)). **Verdict: overkill.** A feature store solves consistent low-latency feature serving across many models with train/serve skew risk. With ~2–3 candidate features (recent CTR, spend velocity, audience overlap), a Postgres table with a scheduled refresh does the same job. No vendor publishes a threshold; general MLOps guidance puts the payoff at 10+ models or point-in-time-correct joins over large offline data — neither applies (**judgment call, UNVERIFIED as a documented rule**).

MLflow vs W&B:

MLFLOW		WEIGHTS & BIASES
License/price	Free, open source, no paid tier	Free: 5 seats / 5GB storage / 1GB Weave ingestion/mo ; Pro from \$60/mo (≤ 10 seats, 100GB, teams < 50 employees); Dedicated \$1,500–5,000/mo minimum (W&B pricing)
True cost	\$150–500/mo self-hosted infra ; 4–8 hrs basic setup, 40+ hrs production-grade — UNVERIFIED (third-party estimate) (comparison)	\$0 on free tier

Verdict: W&B free tier beats self-hosted MLflow once engineer time is counted, for a solo/small team running dozens of runs/month. Flips only on a hard no-third-party-SaaS requirement or on outgrowing the free caps.

5.3 ► You probably don't need ML yet — here's the threshold

The platforms' own optimizers are ML, trained on data you will never have.

- **Meta Advantage+** reallocates budget across ad sets in real time using predicted cost-per-result, purchase value, CTR, and bid competition. Meta's guidance: an ad set needs **~50 optimization events per week** to exit the learning phase and deliver reliably ([Meta Advantage+ budget, learning phase](#)).

- **TikTok Smart+** automates targeting, creative rotation, bidding, and budget. TikTok recommends waiting for **25 conversions** and a 7–10 day learning phase, budgeting $\geq 20\times$ **target CPA daily** ([TikTok Smart+](#)).

The killer argument: those thresholds are what the *platform's own* ML needs to work well. At "one song, then a handful of artists," you will struggle to hit 50 conversions/week **per ad set**. If Meta's optimizer — trained on billions of cross-advertiser events — is operating in a low-confidence regime on your data, **a custom model trained on only your data has strictly less signal**. You cannot out-model Meta using a subset of the information Meta has.

Bandit literature agrees: combinatorial-bandit budget allocation degrades with limited samples and high noise, and faces genuine cold-start problems on new campaigns. Practitioners reach for transfer learning or Bayesian hierarchical models specifically to compensate for low data volume — bandits are not naturally data-efficient ([Adaptive Budget Optimization via Combinatorial Bandits](#), [arXiv 2502.02920](#), [Multi-Task Combinatorial Bandits for Budget Allocation](#), [KDD 2025](#)).

What to do instead, in year one:

INSTEAD OF	DO THIS	WHY
Budget-allocation bandit	Meta Advantage+ / TikTok Smart+	Their optimizers see cross-advertiser data; yours can't
Creative-performance classifier	Bayesian A/B between 2–3 variants + human review	At <30 creatives/mo there's nothing to train on
Audience scoring model	Platform lookalikes + broad targeting	Same data-access argument
Feature store	A Postgres table + a cron refresh	3 features don't need infrastructure
Custom LTV model	Simple regression on CTR/CPA vs spend	Interpretable, and the CI will be wide either way

The trigger to revisit — all of these, sustained, not any one:

- $\geq \$50,000/\text{mo}$ combined Meta+TikTok spend, and
- ≥ 10 concurrently active campaigns, and
- ≥ 30 distinct creatives tested per month, and
- individual ad sets/ad groups **comfortably clearing the platforms' own learning thresholds** (50+ weekly conversions per Meta ad set, 25+ per TikTok ad group) across multiple artists simultaneously.

⚠️ These specific dollar/count figures are a reasoned synthesis from the platform documentation cited above — no vendor publishes them as a rule. Treat as an estimate, not a citation. The *structure* of the argument (you need to clear the platforms' own documented learning thresholds before your own model has anything to learn from) is what's well-founded.

Below that line, custom ML is architecture astronautics. The honest year-one ML answer is: **none**. Spend the engineering time on the cost ledger and the attribution honesty problem instead — those are real, unsolved, and nobody else will solve them for you.

6. Multi-tenancy

6.1 Pattern comparison

PATTERN	COST @ 1 TENANT	COST @ 10 TENANTS	COST @ 100 TENANTS	ISOLATION	OPS COMPLEXITY
Shared schema + RLS	Minimal — one small compute, one schema	Still one instance; cost tracks total data/query volume, not tenant count	Still one instance, but noisy-neighbor risk rises sharply — one tenant's heavy queries degrade everyone	Weakest — relies entirely on correct policy logic; one missing policy leaks cross-tenant data	Lowest — single migration path, but demands rigorous policy testing + indexing discipline
Schema-per-tenant	Slight overhead	Modest — one instance, N schemas; migrations run N times per deploy	Migration orchestration becomes a real burden; comfortable range is " 100s of tenants ", degrading past that without Citus	Strong — needs both a search_path bypass <i>and</i> cross-schema access to leak	Moderate — N-way migration coordination, pooling overhead
DB-per-tenant (traditional RDS/Cloud SQL)	High — a dedicated instance for one customer	Costly — 10 instances x baseline compute/storage/backup	Prohibitive — impractical past ~50 tenants (per-instance patching, backup, migration fleet)	Strongest — instance-level, no shared blast radius	Highest
DB-per-tenant via Neon branching	Near-zero marginal — copy-on-write from base schema, ms provisioning	Cheap — 10 branches fit Launch's included 10; mainly CU-hours when active (compute suspends when idle)	Roughly linear: \$1.50/branch-mo beyond plan allotment + per-branch storage divergence + CU-hours	Strong — connection-string-level isolation	Moderate-high — provisioning is scriptable and cheap, but fleet migrations + control plane are real work

Sources: [PostgreSQL RLS docs](#), [Supabase RLS](#), [Neon RLS](#), [PlanetScale: approaches to tenancy in Postgres](#), [Crunchy Data: designing Postgres for multi-tenancy](#), [Neon: multi-tenancy and database-per-user](#), [Percona: multi-tenants and branches in Neon](#), [Neon pricing](#).

6.2 RLS performance — the thing that actually bites

Supabase's own troubleshooting guide notes RLS overhead "can be massive" on **unindexed policy columns**, and that indexing the policy column (e.g. `auth.uid() = user_id`) can yield a **~100x** improvement. Use `SECURITY DEFINER` helper functions and JWT custom claims to avoid per-row subqueries; measure with `EXPLAIN ANALYZE` and optimize once queries exceed ~50ms ([Supabase RLS performance](#)).

```
-- The pattern that matters: index every tenant_id used in a policy.
ALTER TABLE fact_ad_performance_daily ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON fact_ad_performance_daily
  USING (tenant_id = current_setting('app.tenant_id', true)::uuid);

-- Without this index, every RLS-filtered query seq-scans. With it, ~100x better.
CREATE INDEX ON fact_ad_performance_daily (tenant_id);
```

6.3 Recommendation

Now (handful of artists): shared schema + RLS. At 1–10 tenants, RLS overhead is negligible with basic indexing discipline, and it keeps ops minimal — one schema, one migration path, one compute instance. Your tenants have low, roughly comparable query volume: exactly the profile where noisy-neighbor and leak risk are both low.

Two independent triggers to move:

1. **Performance:** RLS-filtered queries regularly exceed ~50ms *after* indexing, or one label generates disproportionate event volume.
2. **Trust/compliance:** a label's legal team requires contractual data isolation that "we use RLS policies" doesn't satisfy. Increasingly likely with major-label-adjacent clients.

Next pattern: Neon branch-per-tenant — skip schema-per-tenant entirely. Schema-per-tenant exists mainly to dodge the cost of *traditional* DB-per-tenant. Neon's copy-on-write branching removes that cost (near-zero marginal storage, ms provisioning, per-branch compute suspend), so you can jump straight to connection-string-level isolation and stay inside Neon's included branch allotment (10 on Launch, 25 on Scale) well past the "handful of artists" stage ([Neon multi-tenancy](#), [Neon pricing](#)).

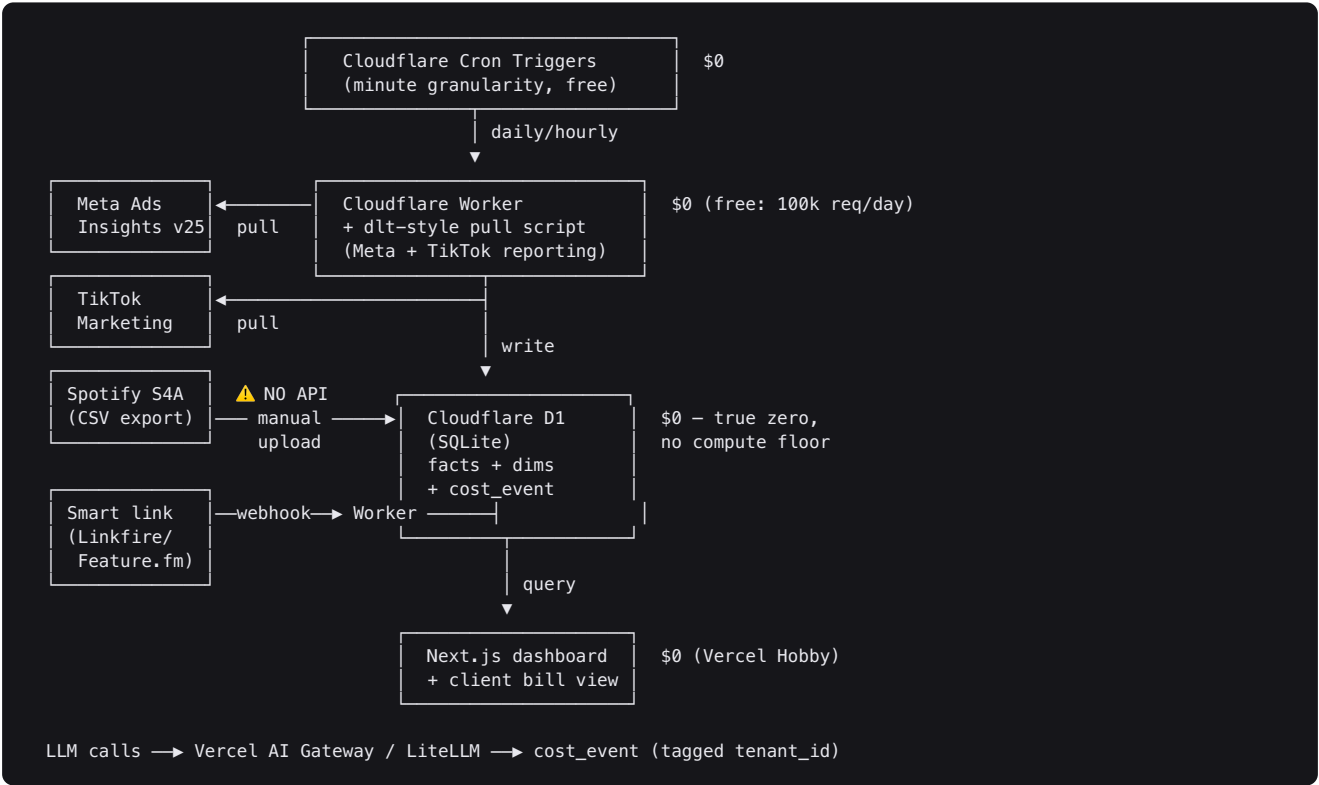
⚠ **Refinement — prefer project-per-tenant over branch-per-tenant.** Neon's own multitenancy guidance recommends **one project per tenant**, not one branch per tenant, on the reasoning that a project is "the logical equivalent of an instance but without the management overhead" — each with independently scaling compute, independent point-in-time recovery, and its own scale-to-zero cycle ([Neon multitenancy](#), [database-per-tenant](#)). Branches are designed as copy-on-write *dev/test copies of one database*, and they inherit their parent project's limits — so branch-per-tenant couples tenants to a shared project boundary and forfeits per-tenant PITR, which is usually the exact thing a label's legal team is asking for.

The project quota is more generous than the branch allotment, which changes the cost picture: **Free and Launch both include 100 projects**; Scale includes **1,000** ([Neon pricing](#)). So project-per-tenant stays inside the included allowance well past dozens of tenants without touching the \$1.50/branch-month overage that branch-per-tenant would start incurring at 11 tenants on Launch. Same note as above applies: Neon **explicitly discourages schema-per-tenant** for SaaS, saying it "doesn't reduce operational overhead compared to database-per-tenant while limiting features like independent point-in-time recovery" — which independently confirms the "skip schema-per-tenant" call above.

The real cost of that move isn't hosting — it's the control plane. Neon's own guidance calls automated onboarding/teardown, fleet-wide migration tooling, and per-tenant monitoring "essential." Budget it as an engineering project, not a line item.

7. Reference architectures

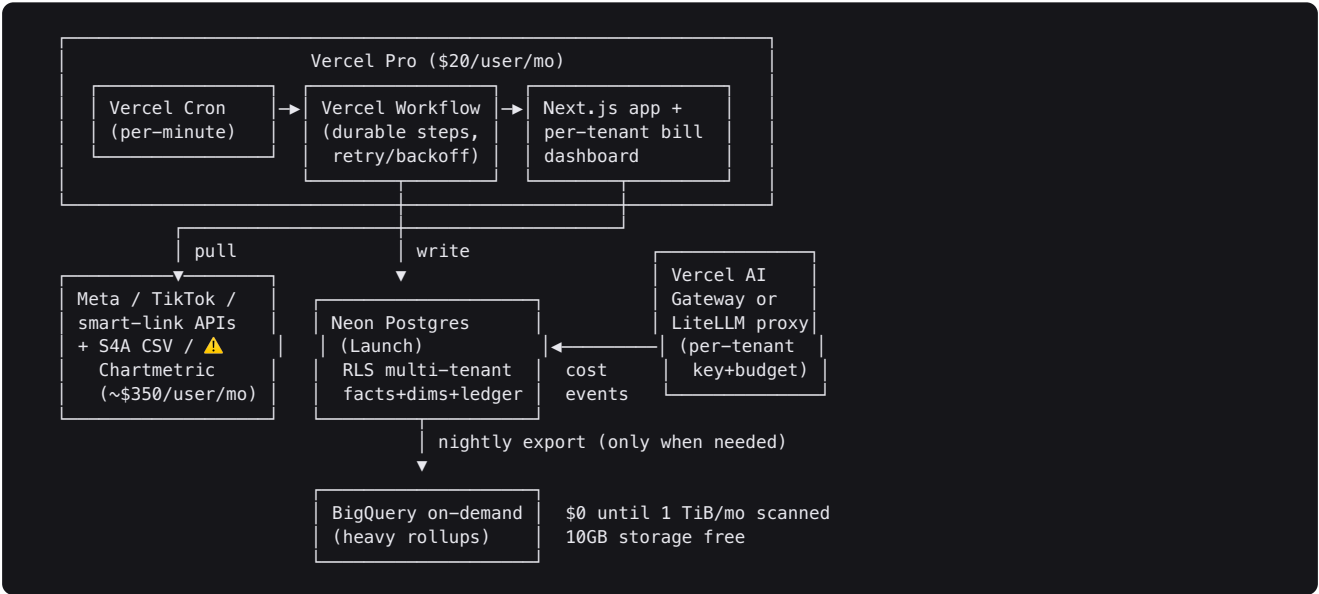
Tier 1 — "\$0-at-idle scrappy" (start here)



Idle cost	\$0/mo — genuinely, if you stay on free tiers
What it costs in practice	\$0–5/mo (Workers Paid only if you exceed 100k req/day)
What breaks	D1 free caps: 5M rows read/day, 100k written/day, 5GB (D1 pricing). Vercel Hobby crons are once-per-day — hence Cloudflare Cron here. No team seats. Analytics queries over D1 get slow past a few million rows
Trigger to move up	You need >1 team member, hourly pulls with Vercel-native tooling, Postgres-specific features (RLS, JSONB ops, window functions at scale), or you cross D1's write cap

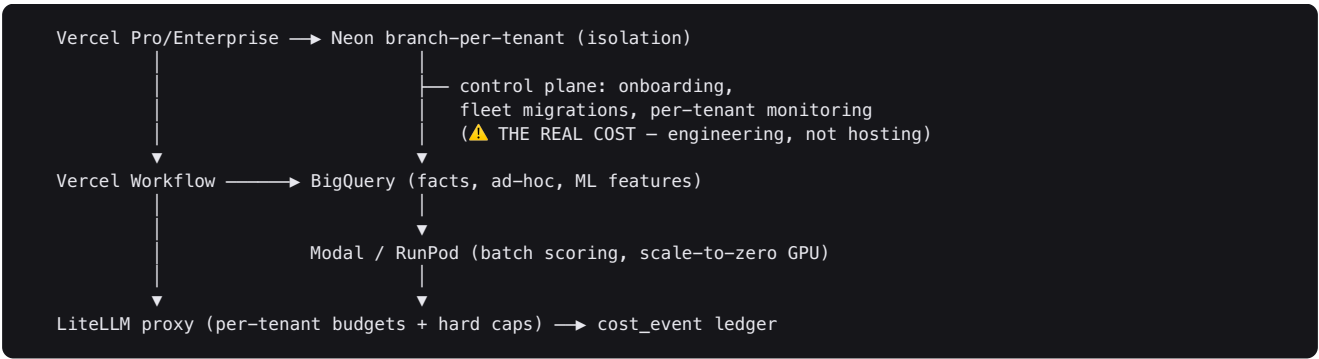
Alternative Tier 1 (Vercel-native): Vercel Hobby + Neon Free + Upstash PAYG. Same \$0, but ⚠️ verify Neon's `check_availability` heartbeat claim (\$1.4) and accept once-per-day crons.

Tier 2 — "Growth" (first real clients)



Idle cost	~\$20–45/mo (Vercel Pro seat + Neon Launch compute/storage). ⚠️ This is the floor the "\$0 serverless" framing hides — it's plan fees, not compute
What it adds	Per-minute crons, durable multi-step pulls, RLS multi-tenancy, team seats, real Postgres
What breaks	Neon's disaggregated storage degrades on big sequential scans (Neon vs Supabase) — keep facts narrow, pre-aggregate. RLS noisy-neighbor risk appears if one label dwarfs the others
Trigger to move up	Rollup queries taking seconds after indexing; a client demanding contractual isolation; sustained ≥\$50k/mo ad spend (the ML threshold)

Tier 3 — "Scale" (only if the triggers actually fire)



Idle cost	~\$45–100+/mo + per-branch fees (\$1.50/branch-mo beyond allotment)
Trigger	All of: ≥10 tenants, contractual isolation demands, ≥\$50k/mo spend clearing platform learning thresholds
The honest warning	Most projects at this scale never need Tier 3. If you find yourself here without the triggers having genuinely fired, you've built for a company you don't have

Migration path

```

Tier 1 —————> Tier 2 —————> Tier 3
$0/mo                $20-45/mo          $45-100+/mo

Triggers:            2nd team member    10+ tenants
                    hourly pulls needed  contractual isolation
                    D1 write caps hit    $50k+/mo ad spend
                    need real Postgres   rollups slow after indexing

What carries over unchanged:
  ✓ the schema ($3.4) — write it Postgres-flavored from day one
  ✓ the cost_event ledger ($4) — the tagging discipline is the asset
  ✓ the dlt pull scripts — same code, different sink

What has to be rebuilt:
  ⚠ D1 → Postgres (SQLite dialect differences; do this at Tier 1-2)
  ⚠ RLS → branch-per-tenant control plane (Tier 2-3, real engineering)

```

Design the Tier 1 schema in Postgres dialect even while running on D1. The migration tax is paid once, at the cheapest possible moment, if you avoid SQLite-only constructs from the start.

What to *not* build — the skeptic's summary

COMPONENT	VERDICT AT OUR SCALE	WHY
Snowplow / Segment / RudderStack	✗ Skip	Solve identity resolution + fan-out at 100M+ events/mo. Segment's MTU model is priced for a different product entirely
Fivetran / Airbyte	✗ Skip	Neither has a Spotify for Artists connector, so you write custom code anyway. Recurring cost for 2 connectors a free script covers
Snowflake / Databricks / ClickHouse Cloud	✗ Skip	Real monthly floors (\$66–\$440+) for GB-scale data. BigQuery on-demand is \$0 and does the same job here
Iceberg / Delta on S3/R2	✗ Skip	Multi-engine petabyte optionality you will never exercise
Airflow / MWAA / Dagster / Temporal	✗ Skip	~\$350/mo (MWAA) or \$100/mo (Temporal) floors to run 3 daily API pulls. Cron is the answer
Feast	✗ Skip	3 features don't need a feature store
Custom ML (bandits, classifiers)	✗ Skip	You cannot out-model Meta with a strict subset of Meta's data. See §5.3
Kafka / Redpanda	✗ Skip	Built for millions of events/sec
A cost ledger table	✓ Build	Nobody sells this for you, and the user explicitly asked for it
Attribution honesty (modeled vs measured)	✓ Build	The stream join is a heuristic. Model it as one
The Postgres schema	✓ Build	Cheap, portable, carries across all three tiers

The single best architectural decision available: use less. A cron job, a Python script, one Postgres database, and a `cost_event` table cover every stated requirement — "every cost tracked," "analytics ready for ingestion and insights," "scale to zero" — at **\$0/mo idle**. Every component above that is a bet on a scale you don't have and may never reach.

Addendum — independent verification pass (July 2026)

A second research pass re-verified this report against live vendor docs. It confirmed the major findings (Supabase Pro never pauses; PlanetScale has no scale-to-zero; Spotify for Artists has no public API; cron beats every persistent orchestrator; Postgres is sufficient for years; no custom ML). It surfaced **two corrections and several additions** worth folding in.

► Corrections to claims above

1. Vercel AI Gateway now HAS enforcing budget caps (§4.2, §4.4 corrected inline). The report previously said AI Gateway was "observability/reporting only" with no enforcement. That is no longer true: per-API-key budgets reject requests once the cap is hit ([Budgets for API keys on AI Gateway](#)).

⚠️ **Precision matters here, and a third verification pass corrected this entry itself.** The changelog's "rejects further requests" reads as a hard cap, but Vercel's own docs page qualify it: *"A budget is a soft cap, not a hard limit. The check runs at the start of each request, so the request that crosses the limit still completes and total spend can end up slightly over the budget"* ([API Key Budgets](#)). **Both descriptions are true and neither alone is accurate:** the budget genuinely enforces (subsequent requests are rejected — categorically different from an alert), but overshoot is bounded at one in-flight request rather than zero. There is also a **1–2 minute window after key creation where the budget is not enforced at all**. For per-tenant LLM budgets this is fine — a one-request overshoot on a sub-dollar call is immaterial — but do not describe it to a client as a hard cap, and do not rely on it for a newly minted key's first two minutes.

This still materially weakens the case for adding LiteLLM — issuing one Gateway key per tenant gives attribution and enforcement from a component already in the stack, without a second proxy hop, a Redis/Postgres dependency, or (see below) a fresh supply-chain surface.

2. Meta's Advantage+ threshold has dropped to 25 conversions/week. §5.3 cites ~50 optimization events/week per ad set. As of 2026, improved signal processing lowered the Advantage+ Shopping threshold to **25 conversions/week** (15 for App campaigns) ([Meta AI automated ads 2026](#), third-party — **UNVERIFIED** against Meta's own docs). **This does not weaken §5.3's conclusion — it sharpens it.** The argument stands on the same logic at either number: below Meta's own stated threshold, the company with the auction and the user graph says it lacks signal, so a custom model on strictly less data is strictly worse.

⚠️ Security: LiteLLM had a supply-chain compromise (March 2026)

§4.2/§4.4 recommend LiteLLM as the enforcement layer without a security caveat. It needs one. Malicious code in **PyPI versions 1.82.7 and 1.82.8** stole cloud credentials, SSH keys, and Kubernetes secrets; a separate pre-auth SQLi CVE was also disclosed ([LiteLLM security update, March 2026](#), [Trend Micro analysis](#)). **Pin ≥1.83.0 and audit before adopting it as a credential-handling gateway.** Combined with correction #1, the recommendation shifts: **use AI Gateway per-key budgets; reach for LiteLLM only if you need multi-provider routing/fallback that AI Gateway doesn't cover.**

Vendor stability — three of the LLM-tooling options changed owners

Relevant to §1.3's gotcha #4 (vendor volatility as an architectural risk for a reusable template):

TOOL	CHANGE	IMPLICATION
Helicone	Acquired by Mintlify (Mar 2026); now in "maintenance mode" — security/bug fixes only, no clear roadmap (Joining Mintlify)	Do not adopt as a core dependency
OpenMeter	Acquired by Kong (Sep 2025), folding into Konnect (Kong)	Roadmap uncertain; already judged overkill above

TOOL	CHANGE	IMPLICATION
Langfuse	ClickHouse acquisition noted in §4.2; core remains MIT and fully self-hostable (pricing)	Still the safest OSS tracing pick

Net effect: of the four third-party LLM cost tools evaluated, two are now in acquisition limbo and one had a supply-chain incident. That is a strong argument for the native AI Gateway path at Tier 1 — fewer moving parts, no third-party credential custody.

► Data-mutability finding — TikTok "Net Cost Delayed" (affects §3.4 ingest logic)

TikTok ships an official metric acknowledging that **spend data can lag up to 11 hours, and the report shows 0 during the delay** before populating with real values (TikTok: Net Cost Delayed). Meta has **no documented equivalent** — a real documentation asymmetry, not just a research gap.

Consequence for the schema: `fact_ad_performance_daily` must **not** be treated as append-only-immutable on first ingest. A pipeline that reads a fresh 0 as a real 0 will corrupt every downstream ROAS and cost-per-click number.

Re-pull a trailing 7-day window on every run and upsert — the existing primary key (`tenant_id, date_key, channel, platform_ad_id, attribution_setting`) already makes this idempotent, so this is an ELT-logic requirement, not a schema change. Consider an `is_finalized BOOLEAN` column so dashboards can visually distinguish provisional days from settled ones.

Sample-size math behind §5.3's threshold

§5.3 flags its dollar/count figures as a reasoned synthesis rather than a citation. Here is the arithmetic that grounds them. For comparing two proportions at $\alpha=0.05$, power=0.80:

```
n_per_arm ≈ 16 · p · (1-p) / δ²      (p = baseline rate, δ = absolute lift)

At p = 2% (a realistic smart-link click→conversion rate), detecting a +20%
relative lift (2.0% → 2.4%, δ = 0.004):

n ≈ 16 × (0.02 × 0.98) / 0.004² ≈ 19,600 clicks per arm → ~392 conversions per arm
```

~19,600 clicks per creative variant to resolve a 20% difference. At a \$0.50 CPC that is ~\$9,800 per arm — roughly \$20k to compare two creatives once.

And bandits do not rescue you — the common belief that they need less data is backwards: "MAB algorithms always require more or equal sample size than an A/B test given the same type I error α , power $1-\beta$, and minimum detectable effect" (GeeksforGeeks, third-party). Bandits reduce *regret while learning* — they earn more during the test — they do not lower the information cost of **knowing**. Some platforms won't activate a bandit below **250 conversions per arm** (UNVERIFIED, third-party). This independently corroborates §5.3's "custom ML is architecture astronautics" verdict from the statistics rather than from the platform-capability argument.

Additions worth noting

- Vercel Fluid "Scale to One" keeps the last production instance warm for up to 14 days **at no idle charge**, materially reducing cold starts without creating a floor (Scale to One). Relevant to §1.1's "no official cold-start number published."
- Meta's Marketing API access tier threshold dropped to 500 API calls in the prior 15 days, making it substantially easier for a small app to qualify (Meta blog). Good news for §2D.
- Fivetran is free to 500,000 MAR, with a \$5 base charge per connection added Jan 2026 once you exceed it (2026 pricing updates). Doesn't change §2D's verdict — the Spotify gap collapses the business case regardless of price.

- **Neon explicitly recommends project-per-tenant**, cautioning that schema-per-tenant *"doesn't reduce operational complexity or costs"*, and claims users run *"hundreds of thousands of projects with just one engineer"* ([Neon multitenancy](#)). This **supports §6.3's "skip schema-per-tenant entirely"** — from the vendor's own mouth. Two caveats: it's Neon recommending more Neon projects, and **Free/Launch cap at 100 projects/org** (Scale: 1,000) ([pricing](#)). The disqualifier for us at Tier 1 stands: this product's thesis is cross-tenant analytics, and project-per-tenant turns a `GROUP BY` into an application-layer fan-out.
- **FinOps Foundation** names three shared-cost allocation strategies — even split, fixed proportional, and **variable proportional** ("more mature organizations rely on variable proportionality") — and advises **"follow the KISS principle"** ([Identifying & allocating shared costs](#)). This is the external backing for §4.3's three-bucket rule. Industry baseline: **18–25% of monthly cloud spend lands unallocated even at mature FinOps shops** ([nOps](#), third-party, **UNVERIFIED**) — useful when setting client expectations about what a per-tenant bill can honestly promise.

Conflicts left unresolved (do not cite externally without checking)

CLAIM	CONFLICT
RudderStack free tier	250K events/mo vs 10K events/mo — both citing rudderstack.com/pricing
Prefect Cloud free tier	1 seat / 5 deployments vs 2 users / 5 workflows. The 500 min/mo compute figure is consistent
Vercel Workflow status	Public beta vs GA as of April 2026 — both citing Vercel's own changelog/blog
TikTok max click-through attribution window	Official page fetched said "1 or 7 day"; third parties claim 14/28-day options exist
Meta <code>action_attribution_windows</code> live enum	§3.2's Jan 2026 deprecation rests on <code>ppc.land</code> citing Meta's Developer Blog; Meta's own reference pages errored on fetch. Confirm against the live API reference before writing ETL defaults — requesting a removed window returns empty data, not an error , which fails silently

Sources

Compute / scale-to-zero

- [Vercel Functions usage and pricing](#)
- [Vercel pricing](#)
- [Cloudflare Workers pricing](#)
- [AWS Lambda pricing](#)
- [AWS VPC pricing \(NAT Gateway\)](#)
- [Understanding and remediating cold starts: an AWS Lambda perspective](#)
- [Modal cold start guide](#)
- [Modal pricing](#)
- [Google Cloud Run pricing](#)
- [Cloud Run: configuring min instances](#)
- [Cloud Run: connecting to a VPC network](#)
- [Configure Serverless VPC Access](#)

- [Fly.io pricing](#)
- [Fly.io cost management](#)

Databases

- [Neon scale-to-zero](#)
- [Neon pricing](#)
- [Neon usage-based pricing](#)
- [Neon latency benchmarks \(community\)](#)
- [Neon discussion #12900 — check_availability heartbeat](#)
- [Supabase free project pausing](#)
- [Supabase pricing](#)
- [PlanetScale pricing](#)
- [Turso pricing](#)
- [Cloudflare D1 pricing](#)
- [DuckDB docs](#)
- [MotherDuck billing](#)
- [MotherDuck pricing](#)
- [Upstash Redis pricing \(docs\)](#)
- [Upstash pricing](#)
- [Neon vs Supabase architecture analysis](#)
- [Tiger Data: the Postgres optimization treadmill](#)
- [VeloDB: when to scale PostgreSQL analytics](#)

Ingestion / warehouse / ELT / orchestration

- [PostHog pricing](#) · [PostHog self-host](#)
- [Jitsu pricing](#)
- [RudderStack pricing](#)
- [Segment MTUs and throughput](#)
- [Vercel Queues pricing](#)
- [AWS SQS pricing](#)
- [Cloudflare Queues pricing](#)
- [Redpanda Serverless](#)
- [Google BigQuery pricing](#)
- [Databricks SQL pricing](#)
- [Dremio: Apache Iceberg vs Delta Lake](#)
- [Airbyte: Facebook Marketing source](#)
- [Airbyte: TikTok Marketing source](#)
- [Airbyte discussion #36110 — Spotify connector request](#)
- [Airbyte pricing](#)
- [Fivetran: Facebook Ads connector](#)
- [Fivetran: TikTok Ads connector](#)

- [Fivetran usage-based pricing](#)
- [dlt: Facebook Ads verified source](#)
- [dlt: TikTok Business](#)
- [Meltano Hub: tap-facebook · tap-tiktok](#)
- [Spotify: exporting data from Spotify for Artists](#)
- [Chartmetric API docs](#)
- [Vercel Cron usage and pricing · 100 crons per project changelog](#)
- [Vercel Workflows pricing](#)
- [Prefect pricing](#)
- [Dagster pricing · Dagster Solo/Starter pricing updates, May 2026](#)
- [Temporal pricing](#)
- [AWS MWAA pricing](#)

Meta / TikTok platform data models

- [Meta Ads Insights API](#)
- [Meta ad campaign structure](#)
- [Meta: About ThruPlay](#)
- [Meta: Clicks \(All\) vs Link Clicks](#)
- [Meta: Unique link clicks](#)
- [Introducing Graph API v25.0 and Marketing API v25.0](#)
- [ppc.land: Meta restricts attribution windows and data retention](#)
- [TikTok: Marketing API overview](#)
- [TikTok: Attribution overview](#)
- [TikTok: About the attribution window](#)
- [TikTok: About engaged view-through attribution](#)
- [TikTok: Video play metrics](#)
- [TikTok: Basic metrics](#)
- [TikTok: About the reach metric](#)
- [TikTok Business API SDK — ReportingApi](#)
- [Linkfire: retargeting parameters](#)
- [Feature.fm: conversion data · Feature.fm Partners API](#)
- [ToneDen FanLinks API](#)

ML

- [Modal pricing · Replicate pricing · RunPod serverless pricing · Baseten pricing · Beam pricing](#)
- [Feast docs · Feast GitHub](#)
- [Weights & Biases pricing](#)
- [Meta Advantage+ budget](#)
- [TikTok: About Smart+ campaigns](#)
- [Adaptive Budget Optimization via Combinatorial Bandits \(arXiv 2502.02920\)](#)
- [Multi-Task Combinatorial Bandits for Budget Allocation \(KDD 2025\)](#)

Multi-tenancy

- [PostgreSQL row security policies](#)
- [Supabase RLS · Supabase RLS performance and best practices](#)
- [Neon RLS · Neon: multi-tenancy and database-per-user design](#)
- [PlanetScale: approaches to tenancy in Postgres](#)
- [Crunchy Data: designing your Postgres database for multi-tenancy](#)
- [Percona: multi-tenants and branches in Neon serverless PostgreSQL](#)

FinOps / cost attribution

- [Vercel Spend Management · Improved hard caps for Spend Management · Spend Management: realtime usage alerts, SMS, project pausing](#)
- [Vercel: unpause a project \(REST API\)](#)
- [Vercel: manage and optimize usage](#)
- [Vercel AI Gateway: Custom Reporting · Observability and spend · AI Gateway pricing](#)
- [AWS Cost Explorer pricing · Using the AWS Cost Explorer API · ListCostAllocationTags](#)
- [Cloudflare GraphQL Analytics API · D1 billing · D1 metrics and analytics · Workers metrics and analytics](#)
- [LiteLLM: budgets and rate limits · LiteLLM virtual keys · LiteLLM budget/rate-limit tiers](#)
- [Langfuse pricing · Langfuse self-hosted pricing](#)
- [Helicone pricing · Helicone cost tracking cookbook · Helicone GitHub](#)
- [OpenMeter pricing · OpenMeter GitHub](#)
- [Meta: campaign spend limits available via API · Meta: about daily spending limits](#)

Addendum — July 2026 verification pass

- [Vercel: Budgets for API keys on AI Gateway — the correction to §4.2](#)
- [Vercel: Scale to One — how Fluid solves cold starts](#)
- [LiteLLM security update, March 2026 · Trend Micro: inside the LiteLLM supply chain compromise](#)
- [Helicone: joining Mintlify · Kong acquires OpenMeter · OpenMeter: joining Kong](#)
- [TikTok: About the Net Cost Delayed metric — the 11-hour spend lag; affects ELT upsert logic](#)
- [Meta: updates to Ads Management standard access — 500-call access-tier threshold](#)
- [Fivetran 2026 pricing updates — 500k MAR free; \\$5/connection base](#)
- [Neon: multitenancy guide — vendor's own project-per-tenant recommendation + 100-project cap](#)
- [FinOps Foundation: identifying and allocating shared costs — backing for the three-bucket rule](#)
- [nOps: cloud cost allocation tools — 18–25% unallocated baseline \(third-party, UNVERIFIED\)](#)
- [GeeksforGeeks: A/B testing vs multi-armed bandits — bandits need \$\geq\$ the sample size of an A/B test](#)
- [SplitMetrics: calculating sample size for A/B testing](#)
- [digitalapplied: Meta AI automated ads 2026 — 25-conversion Advantage+ threshold \(third-party, UNVERIFIED\)](#)
- [TikTok Smart+ blog · segwise: TikTok Smart+ 2026 upgrade](#)